



كلية العلوم
السملاية - مراكش
FACULTÉ DES SCIENCES
SEMLALIA - MARRAKECH



Pr. SOUKAINA MJAHER

s.mjahed@uca.ac.ma

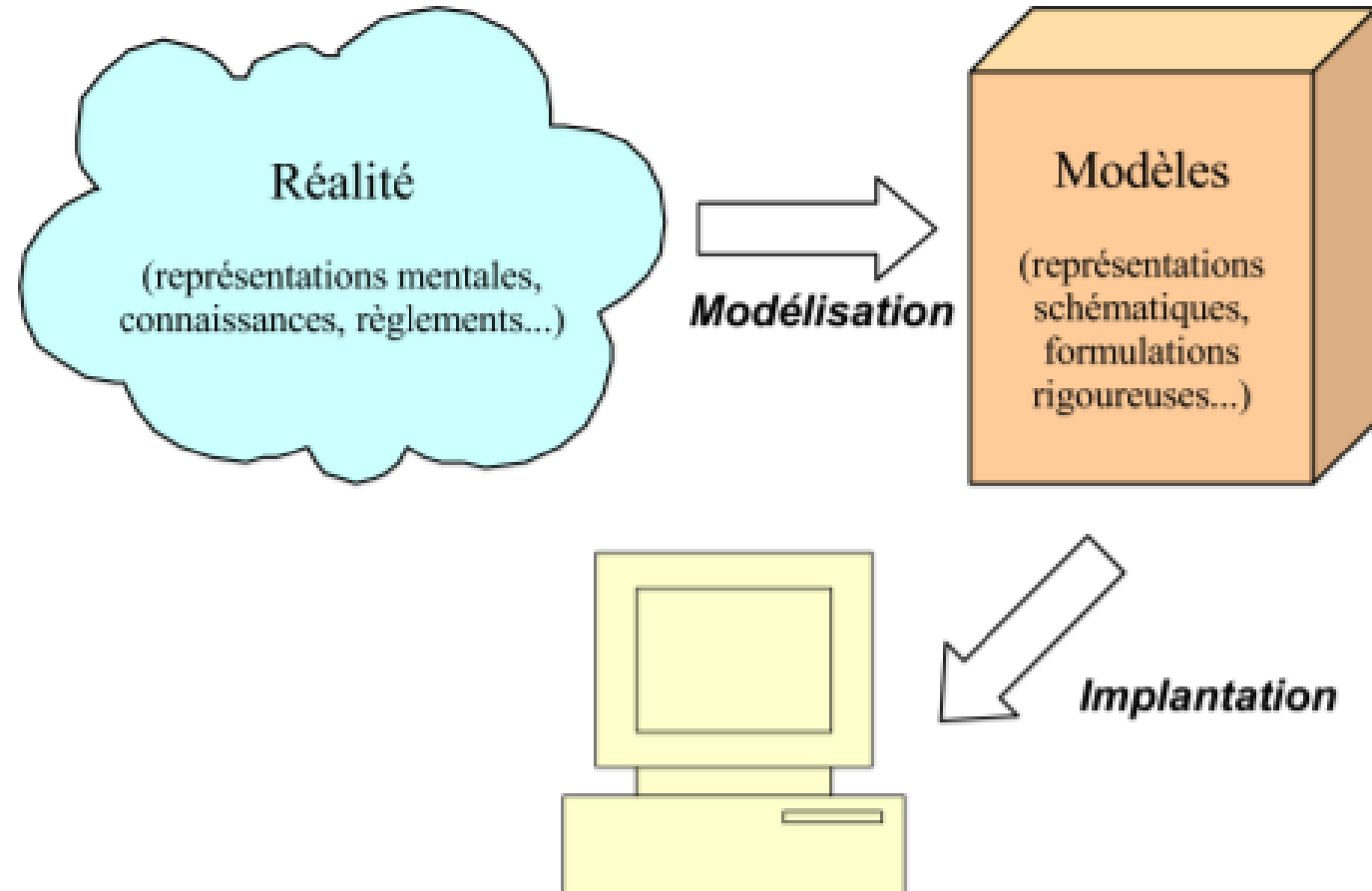
UML – Introduction

- ❑ UML (Unified Modeling Language) est un langage graphique standardisé pour **modéliser**, visualiser, spécifier et documenter des systèmes **logiciels** (et parfois non-logiciels). Il facilite la communication entre développeurs, analystes, architectes et parties prenantes.
- ❑ Objectifs :
 - Clarifier les idées avant d'implémenter.
 - Communiquer la structure et le comportement du système.
 - Servir de documentation vivante et d'artefact pour la conception.
 - Aider à la conception orientée objet et à l'ingénierie des exigences.
- ❑ UML focalise sur l'approche **orientée objet** des applications et données.

UML – Notions de base

❑ **Modèle :**

- Représentation abstraite de la réalité qui exclut certains détails du monde réel.
- Permet de réduire la complexité d'un phénomène en éliminant les détails qui n'influencent pas son comportement de manière significative.
- Il reflète ce que le concepteur croit important pour la compréhension et la prédiction du phénomène modélisé, les limites du phénomène modélisé dépendent des objectifs du modèle.



UML – Notions de base

- ❑ **Maître d'ouvrage (MOA)** : personne morale (entreprise, direction, etc.) entité de l'organisation. Ce n'est jamais une personne.
- ❑ **Maître d'œuvre (MOE)** : personne morale (entreprise, direction, etc.) garante de la bonne réalisation technique des solutions. Il a, lors de la conception du SI, un devoir de conseil vis-à-vis du MOA, car le SI doit tirer le meilleur parti des possibilités techniques.
- ❑ **Le MOA est client du MOE** à qui il passe commande d'un produit nécessaire à son activité.
- ❑ **Le MOE fournit ce produit.** Soit il le réalise lui-même, soit il passe commande à un ou plusieurs fournisseurs (« entreprises ») qui élaborent le produit sous sa direction.
- ❑ **La relation MOA et MOE est définie par un contrat** qui précise leurs engagements mutuels.
- ❑ Le MOE contrôle la qualité technique du produit en assurant le respect des délais fixés par le MOA et en minimisant les risques.
- ❑ **Le MOE est responsable de la qualité technique de la solution.** Il doit, avant toute livraison au MOA, procéder aux vérifications nécessaires (« recette de tests »).

UML – Notions de base

❑ Logiciel :

- Ensemble de programmes, qui permet à un ordinateur ou à un système informatique d'assurer une tâche ou une fonction en particulier.
- Les logiciels, suivant leur taille, peuvent être développés par une personne seule, une petite équipe, ou un ensemble d'équipes coordonnées.
- Des études ont été faites sur les causes de succès et d'échec des projets : la plupart des échecs proviennent non de l'informatique, mais de la maîtrise d'ouvrage (i.e. le client).
- Pour ces raisons, le développement de logiciels dans un contexte professionnel suit souvent des règles strictes encadrant la conception et permettant le travail en groupe et la maintenance du code.
- Ainsi, une nouvelle discipline est née : le génie logiciel.

UML – Notions de base

□ Génie logiciel :

- Domaine de recherche qui a pour objectif de répondre à un problème qui s'énonçait en deux constatations : Fiabilité du logiciel, Délais (et donc coûts) prévus des logiciels satisfaisant le cahier des charges.
- La maintenance est devenue une facette très importante du cycle de vie d'un logiciel.
- Une enquête effectuée auprès de 55 entreprises révèle que 53% du budget total d'un logiciel est affecté à la maintenance. Ce coût est réparti comme suit :
 - 34% maintenance évolutive (modification des spécifications initiales) ;
 - 10% maintenance adaptative (nouvel environnement, nouveaux utilisateurs) ;
 - 17% maintenance corrective (correction des bugs) ;
 - 16% maintenance perfective (améliorer les performances sans changer les spécifications) ;
 - 6% assistance aux utilisateurs ;
 - 6% contrôle qualité ;
 - 7% organisation/suivi ;
 - 4% divers.

UML – Notions de base

❑ Génie logiciel :

- Pour apporter une réponse à tous ces problèmes, le génie logiciel s'intéresse particulièrement à la manière dont le code source d'un logiciel est spécifié puis produit.
- Ainsi, le génie logiciel touche au cycle de vie des logiciels :
 - l'analyse du besoin,
 - l'élaboration des spécifications,
 - la conceptualisation,
 - le développement,
 - la phase de test,
 - la maintenance.

UML – Notions de base

❑ Qualité du logiciel :

- Divers travaux ont mené à la définition de la qualité du logiciel en termes de facteurs, qui dépendent, entre autres, du domaine de l'application et des outils utilisés.
- **Validité** : aptitude d'un produit logiciel à remplir exactement ses fonctions, définies par le cahier des charges et les spécifications.
- **Fiabilité ou robustesse** : aptitude d'un produit logiciel à fonctionner dans des conditions anormales.
- **Extensibilité (maintenance)** : facilité avec laquelle un logiciel se prête à sa maintenance, c'est-à-dire à une modification ou à une extension des fonctions qui lui sont demandées.
- **Réutilisabilité** : aptitude d'un logiciel à être réutilisé, en tout ou en partie, dans de nouvelles applications.
- **Compatibilité** : facilité avec laquelle un logiciel peut être combiné avec d'autres logiciels.

UML – Notions de base

❑ Qualité du logiciel :

- Divers travaux ont mené à la définition de la qualité du logiciel en termes de facteurs, qui dépendent, entre autres, du domaine de l'application et des outils utilisés.
- **Efficacité** : Utilisation optimale des ressources matérielles.
- **Portabilité** : facilité avec laquelle un logiciel peut être transféré sous différents environnements matériels et logiciels.
- **Vérifiabilité** : facilité de préparation des procédures de test.
- **Intégrité** : aptitude d'un logiciel à protéger son code et ses données contre des accès non autorisés.
- **Facilité d'emploi** : facilité d'apprentissage, d'utilisation, de préparation des données, d'interprétation des erreurs et de rattrapage en cas d'erreur d'utilisation.
- Ces facteurs sont parfois contradictoires, le choix des compromis doit s'effectuer en fonction du contexte.

UML – Notions de base

❑ Approche orientée objet :

- Considère le logiciel comme une collection d'objets dissociés, identifiés et possédant des caractéristiques.
- Une caractéristique est soit un attribut, soit une entité comportementale de l'objet (i.e. une fonction).
- La fonctionnalité du logiciel émerge alors de l'interaction entre les différents objets qui le constituent.
- L'une des particularités de cette approche est qu'elle rapproche les données et leurs traitements associés au sein d'un unique objet.

UML – Notions de base

❑ Approche orientée objet :

- Un objet est caractérisé par plusieurs notions :
 - L'identité : qui permet de le distinguer des autres objets.
 - Les attributs : données caractérisant l'objet. Ce sont des variables stockant des informations sur l'objet.
 - Les méthodes : méthodes d'un objet caractérisant son comportement, c'est-à-dire l'ensemble des actions (appelées opérations) que l'objet est à même de réaliser. Ces opérations permettent de faire réagir l'objet aux sollicitations extérieures (ou d'agir sur les autres objets). De plus, les opérations sont étroitement liées aux attributs, car leurs actions peuvent dépendre des valeurs des attributs, ou bien les modifier.
- Représentation abstraite, sous forme d'objets, d'entités ayant une existence matérielle ou bien virtuelle.
- La Conception Orientée Objet (COO) est la méthode qui conduit à des architectures logicielles fondées sur les objets du système, plutôt que sur la fonction qu'il est censé réaliser.

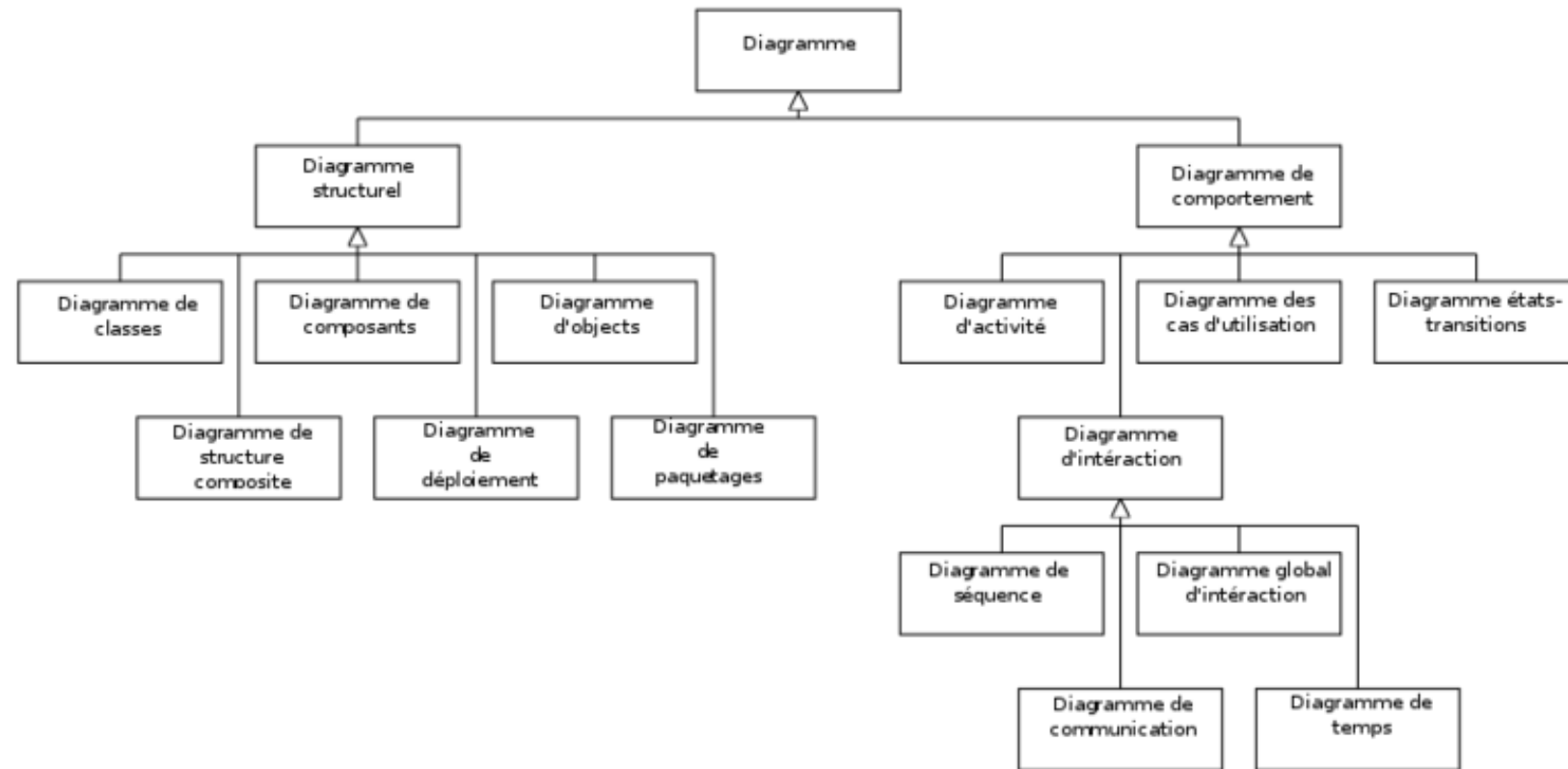
UML – Notions de base

❑ Approche fonctionnelle VS Approche orientée objet :

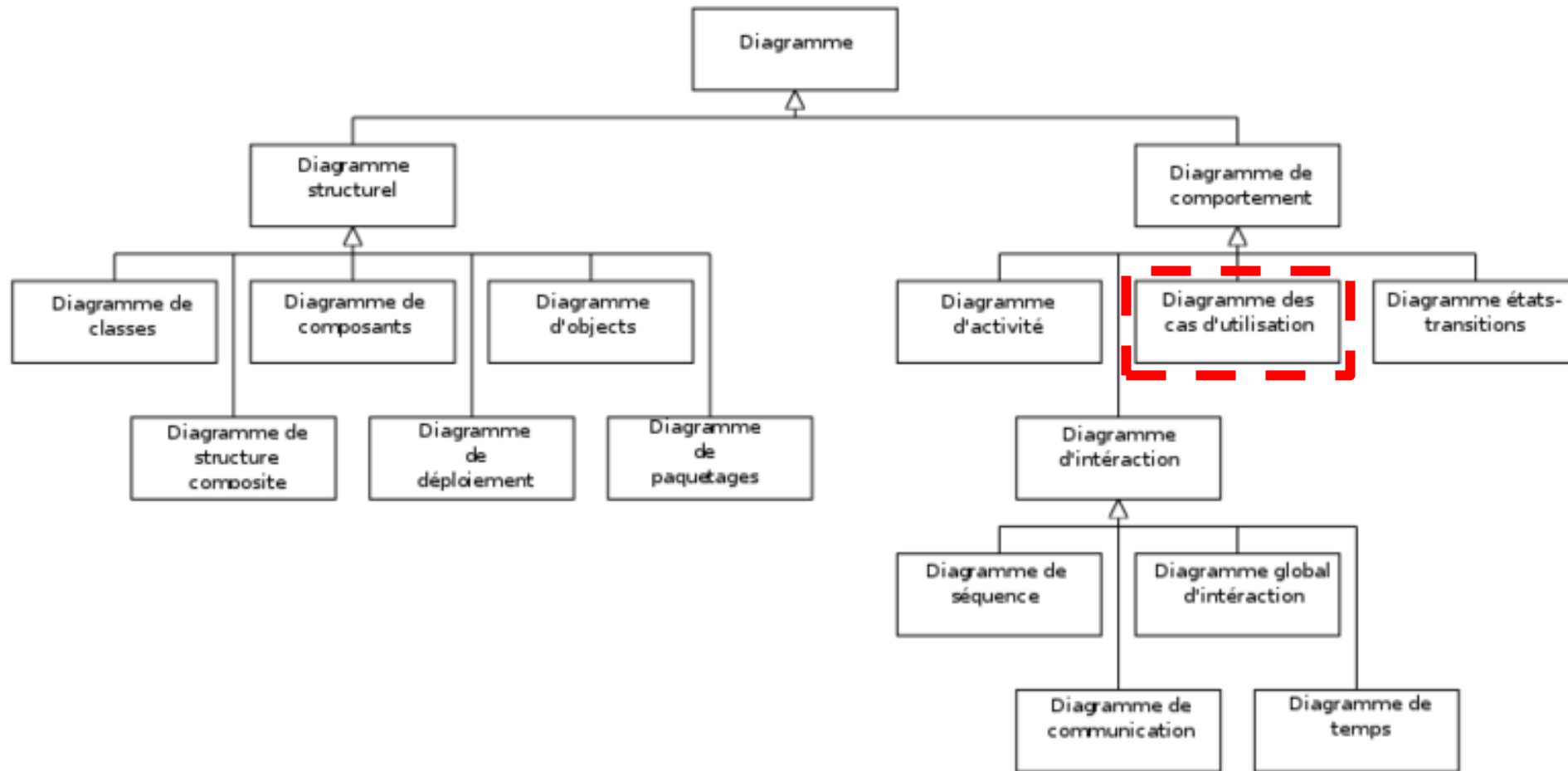
- Tout ce que l'on fait avec un langage de programmation par objets pourrait être fait en programmation impérative. La différence entre une approche fonctionnelle et une approche objet n'est donc pas d'ordre logique, mais pratique.
- **L'approche objet** est une approche orientée donnée. Dans cette approche, les fonctions se déduisent d'un regroupement de champs de données formant une entité cohérente, logique, tangible et surtout stable quant au problème traité.
- **L'approche structurée** classique privilégie une organisation des données postérieure à la découverte des grandes, puis petites fonctions qui les décomposent, l'ensemble constituant les services qui répondent aux besoins.

UML – Définitions

- ❑ Langage graphique qui permet de représenter et de communiquer les divers aspects d'un système d'information. Aux graphiques sont bien sûr associés des textes qui expliquent leur contenu.
- ❑ Permet de donner sur un système une idée utilisable en pratique sans risque d'erreur grave.
- ❑ UML 2.0 comporte plusieurs types de diagrammes représentant des concepts particuliers du système d'information. Ils se répartissent en deux grands groupes.



UML – Définitions



UML – Diagramme des cas d'utilisation

- ❑ Permet de représenter les fonctions d'un système du point de vue de l'utilisateur (appelé « acteur » en UML).
- ❑ Cet acteur ne doit pas nécessairement être un utilisateur humain. Le rôle peut également être attribué à un système externe qui accède à un autre système.
- ❑ **Le diagramme de cas d'utilisation montre en fait la relation entre un acteur et ses demandes ou attentes vis-à-vis du système, sans décrire les actions en cours ni les mettre dans un ordre logique.**
- ❑ **Permet de visualiser facilement et clairement quels cas d'utilisation doivent être pris en compte dans la conception pour que les acteurs (et au sens large également l'opérateur ou le client) atteignent les objectifs escomptés – sans tenir compte dans un premier temps de la faisabilité technique.**

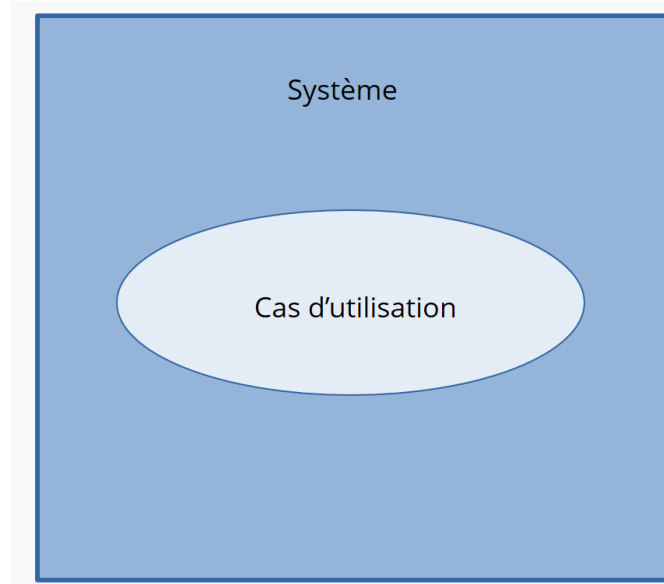
UML – Diagramme des cas d'utilisation

- ❑ **Acteur :**
- ❑ Les **acteurs principaux** : vont réaliser le cas d'utilisation.
- ❑ Les **acteurs secondaires** : font que recevoir des informations a l'issue de la réalisation d'un cas d'utilisation.
- ❑ Il ne peut y avoir qu'un seul acteur principal par cas d'utilisation.
- ❑ Les stéréotypes « primary » ou « secondary » indique si l'acteur est principal ou secondaire pour un cas d'utilisation.



UML – Diagramme des cas d'utilisation

- ❑ **Système** : le système auquel se rapporte le cas d'utilisation est représenté par un rectangle.
- ❑ **Cas d'utilisation** : le cas d'utilisation lui-même est représenté par une ellipse, dans laquelle il y a généralement une courte phrase décrivant le processus et commençant par un verbe.



UML – Diagramme des cas d'utilisation

❑ **Association** : ligne continue entre un acteur et un cas d'utilisation qui indique clairement que l'acteur et le cas d'utilisation décrits dans l'ellipse sont liés.



❑ **Association d'inclusion** (include) : le cas d'utilisation d'où part la ligne de connexion en pointillés inclut un deuxième cas d'utilisation, pointé par la pointe de flèche.

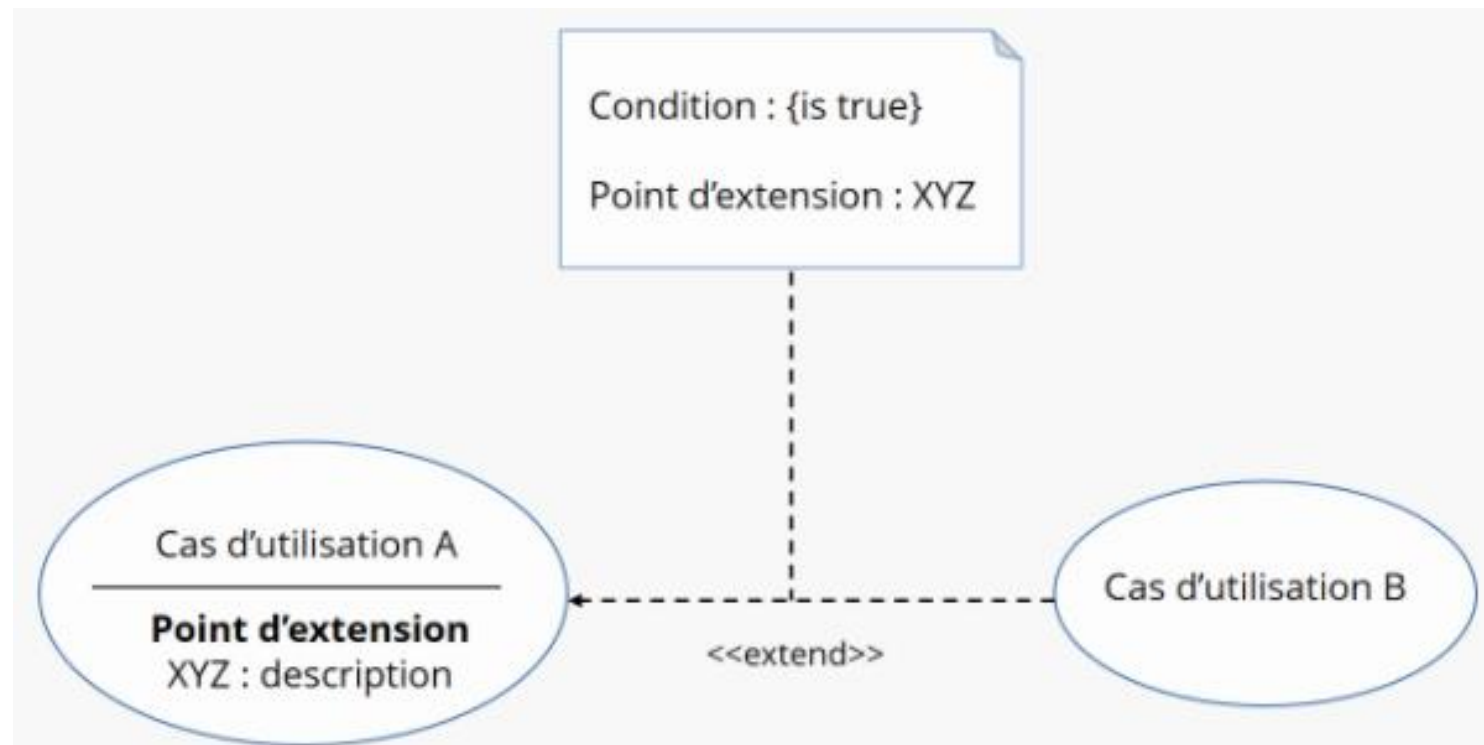


❑ **Association d'extension** (extend) : le cas d'utilisation d'où part la ligne de connexion en pointillés commence peut, sous certaines conditions, étendre le cas d'utilisation pointé par la tête de flèche. Mais ce n'est pas toujours le cas.



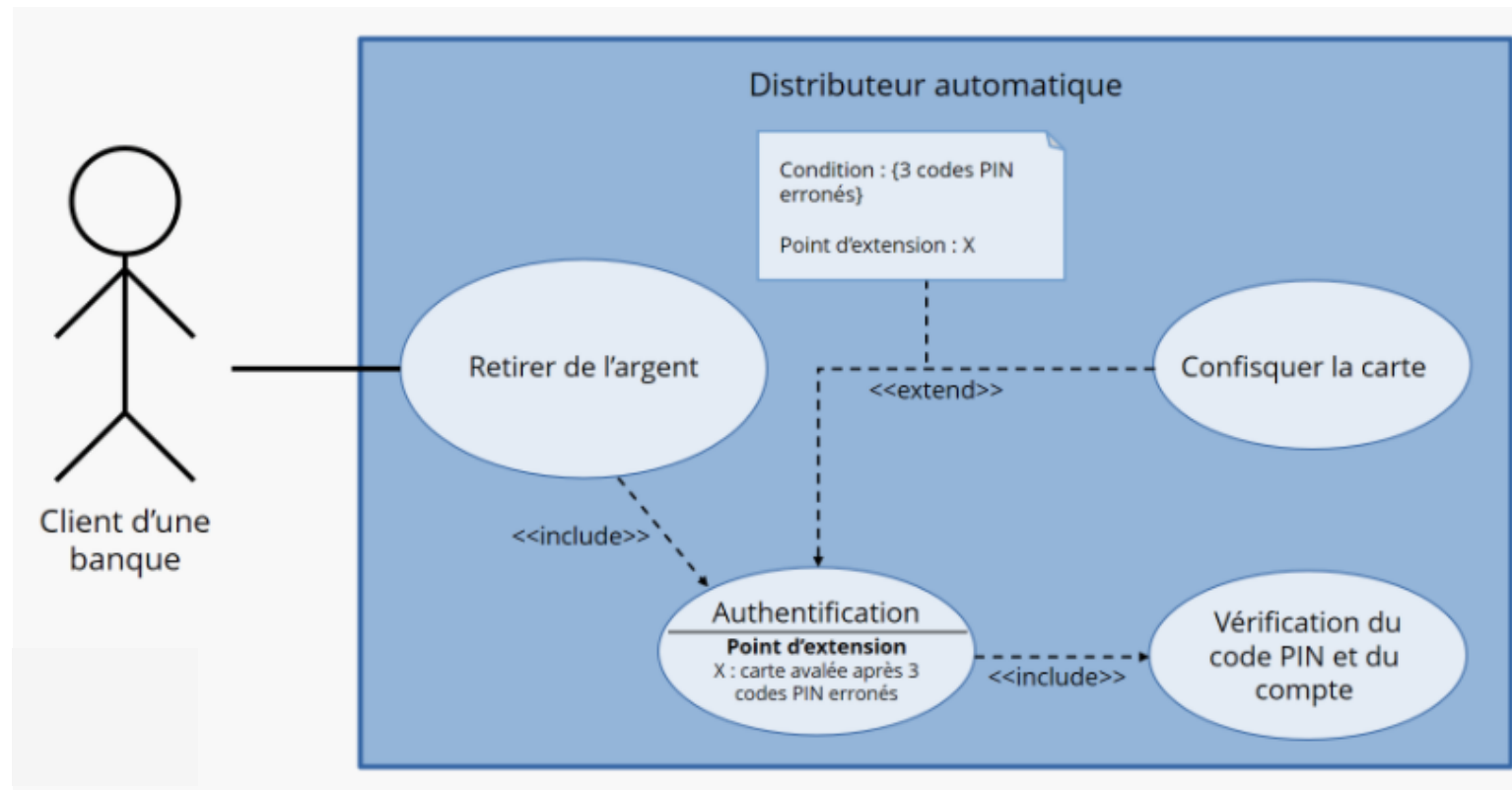
UML – Diagramme des cas d'utilisation

- ❑ **Point d'extension** (extend) : La prise en compte du deuxième cas d'utilisation, dans l'association d'extension, dépend de certaines conditions.



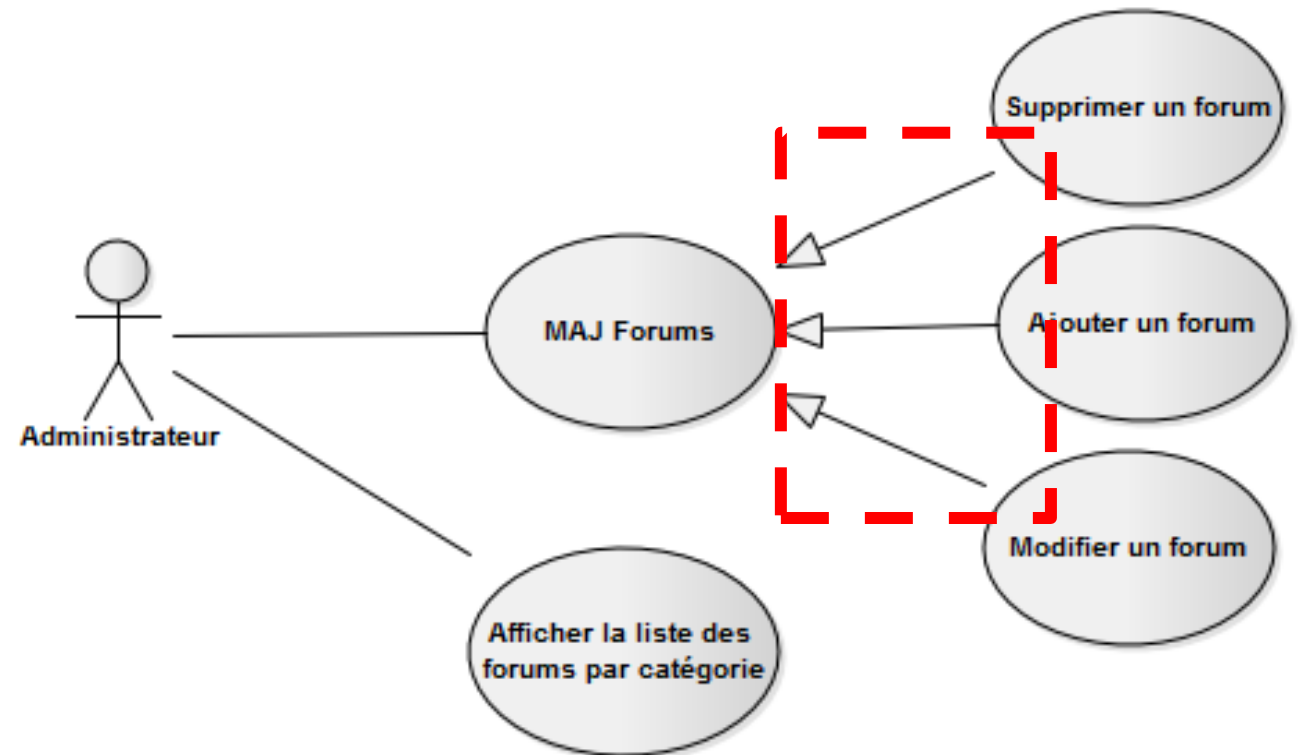
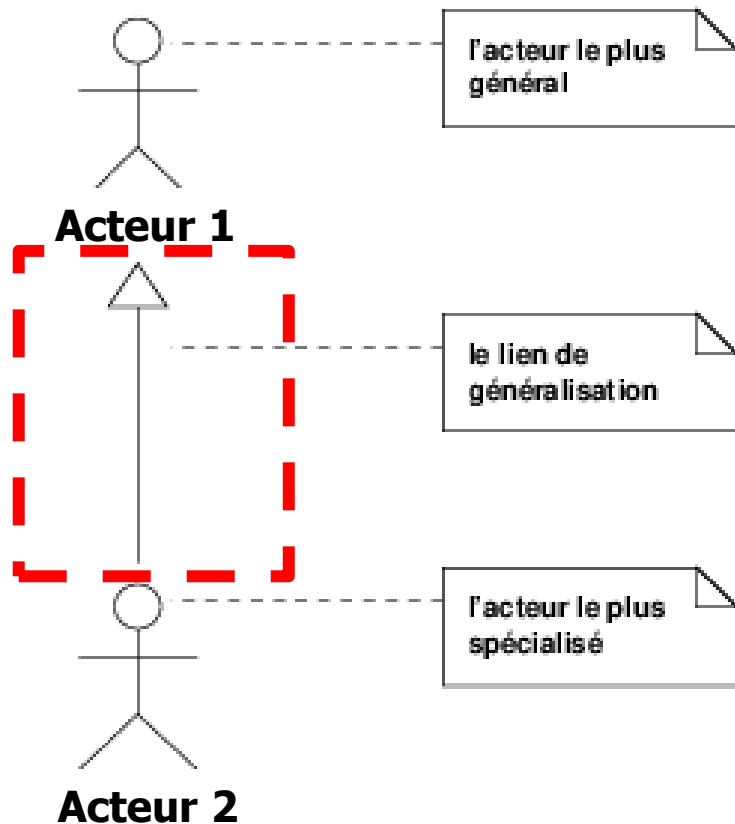
UML – Diagramme des cas d'utilisation

- ❑ **Point d'extension** (extend) : La prise en compte du deuxième cas d'utilisation, dans l'association d'extension, dépend de certaines conditions.



UML – Diagramme des cas d'utilisation

❑ **Généralisation** : utilisé entre acteurs mais aussi entre cas d'utilisation.



UML – Diagramme des cas d'utilisation

❑ Exercice d'application

Une bibliothèque municipale souhaite informatiser la gestion de l'emprunt de livres.

Un usager peut :

- Consulter le catalogue en ligne.
- S'inscrire comme membre.
- Emprunter un livre s'il est membre et que le livre est disponible.
- Retourner un livre.

Un bibliothécaire peut :

- Enregistrer l'inscription d'un nouvel usager.
- Enregistrer les emprunts et les retours de livres.
- Mettre à jour le catalogue (ajouter/supprimer des livres).

1. Identifier les acteurs principaux et secondaires.
2. Lister les cas d'utilisation (use cases) à partir des actions décrites.
3. Dessiner le diagramme de cas d'utilisation UML en montrant :
 - ❖ Les acteurs (avec les bonhommes)
 - ❖ Les cas d'utilisation (ovales)
 - ❖ Les relations (lignes d'association, éventuels liens « include » ou « extend » si nécessaire).

UML – Description textuelle des cas d'utilisation

- ❑ Le diagramme de cas d'utilisation décrit les grandes fonctions d'un système du point de vue des acteurs => N'expose pas de façon détaillée le dialogue entre les acteurs et les cas d'utilisation.
- ❑ Bien que de nombreux diagrammes d'UML permettent de décrire un cas, il est recommandé de rédiger une description textuelle, car c'est une forme souple qui convient dans bien des situations.
- ❑ Une description textuelle couramment utilisée se compose de trois parties.

UML – Description textuelle des cas d'utilisation

☐ Partie 1 : **Identification**

☐ Permet d'identifier le cas.

☐ Doit contenir les informations suivantes :

- **Nom** : utiliser une tournure à l'infinitif (ex. : Réceptionner un colis).
- **Objectif** : une description résumée permettant de comprendre l'intention principale du cas d'utilisation.
- **Acteurs principaux** : ceux qui vont réaliser le cas d'utilisation.
- **Acteurs secondaires** : ceux qui ne font que recevoir des informations à l'issue de la réalisation du cas d'utilisation.
- **Dates** : les dates de création et de mise à jour de la description courante.
- **Responsable** : le nom des responsables.
- **Version** : le numéro de version.

UML – Description textuelle des cas d'utilisation

☐ Partie 2 : **Description des scénarios**

- ☐ Description du fonctionnement du cas sous la forme d'une séquence de messages échangés entre les acteurs et le système.
- ☐ Contient toujours une **séquence nominale** qui décrit le déroulement normal du cas.
- ☐ À la séquence nominale s'ajoutent fréquemment des **séquences alternatives** (des embranchements dans la séquence nominale) et des **séquences d'exceptions** (qui interviennent quand une erreur se produit).

UML – Description textuelle des cas d'utilisation

□ Partie 2 : Description des scénarios

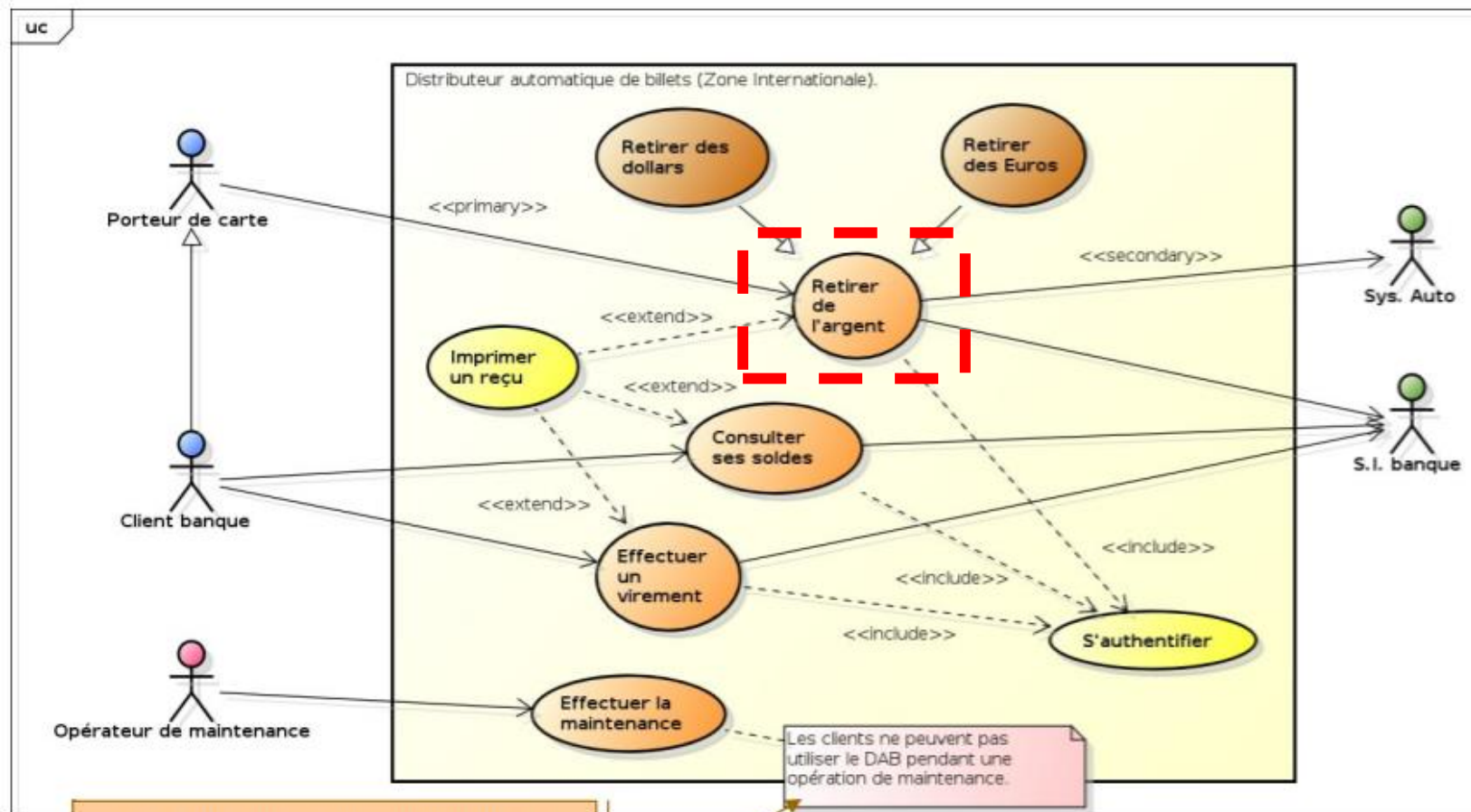
- **Les préconditions** : elles décrivent dans quel état doit être le système avant que ce cas d'utilisation puisse être déclenché.
- **Des scénarios** : ces scénarios sont décrits sous la forme d'échanges d'événements entre l'acteur et le système. On distingue le **scénario nominal** (qui se déroule quand il n'y a pas d'erreur), des **scénarios alternatifs** (qui sont les variantes du scénario nominal) et enfin les **scénarios d'exception** (qui décrivent les cas d'erreurs).
- **Des postconditions** : elles décrivent l'état du système à l'issue des différents scénarios.

UML – Description textuelle des cas d'utilisation

- ❑ **Partie 3 : Exigence non fonctionnelle**
- ❑ Rubrique optionnelle.
- ❑ Contient généralement des spécifications non fonctionnelles (spécifications techniques...).
- ❑ Peut éventuellement contenir une description des besoins en termes d'interface graphique.

UML – Description textuelle des cas d'utilisation

❑ Exemple de description textuelle : Le cas d'utilisation 'Retirer de l'argent'



En UML une note (un commentaire) est représentée par un rectangle dont l'un des coins est retourné. La note est reliée à l'élément ou aux éléments qu'elle décrit par une ou plusieurs lignes pointillées.

UML – Description textuelle des cas d'utilisation

❑ Exemple de description textuelle : Le cas d'utilisation **'Retirer de l'argent'**

Nom	Retirer de l'argent
Objectif	Ce cas d'utilisation permet aux possesseurs de carte bancaire de retirer de l'argent.
Acteurs principaux	Un porteur de carte bancaire.
Acteurs secondaires	Le Système d'Information de la banque et le Système d'Autorisation Globale Carte Bancaire.
Dates	03/01/2022
Responsable	S. Mjahed
Version	1.0
Pré-conditions	- Le DAB contient des billets. - Les connexions avec le Système d'Autorisation et le Système d'information de la banque sont opérationnelles.

UML – Description textuelle des cas d'utilisation

❑ Exemple de description textuelle : Le cas d'utilisation 'Retirer de l'argent'

Scénario nominal

1. Le Porteur de carte introduit sa carte dans le DAB.
2. Le DAB vérifie que la carte introduite est bien une carte bancaire.
3. Le DAB demande le code de la carte au Porteur de carte.
4. Le Porteur de carte saisit son code.
5. Le DAB compare ce code avec celui qui est codé sur la carte.
6. Le DAB demande une autorisation au Système Globale d'autorisation.
7. Le Système d'Autorisation globale donne son accord et indique le crédit hebdomadaire.
8. Le DAB demande le montant désiré au Porteur de carte.
9. Le Porteur de carte saisit le montant.
10. Le DAB vérifie si le montant demandé est inférieur ou égale au crédit hebdomadaire.
11. Le DAB demande au Porteur de carte s'il désire un ticket.
12. Le Porteur de carte accepte le ticket.
13. Le DAB rend la carte et demande au Porteur de carte de la retirer.
14. Le Porteur de carte reprend sa carte.
15. Le DAB délivre le ticket et les billets.
16. Le Porteur de carte prend les billets et le ticket.

UML – Description textuelle des cas d'utilisation

Exemple de description textuelle : Le cas d'utilisation ‘Retirer de l’argent’

Scénario alternatif SA1	SA1 commence au point 5 du scénario nominale. Le DAB indique que le code est erroné. Le DAB enregistre l'échec. Le scénario reprend au point 3 du scénario nominal.
Scénario alternatif SA2	SA2 commence au point 10 du scénario nominale. Le DAB affiche le montant max et demande au Porteur de carte de ressaisir un montant. Le scénario reprend au point 9 du scénario nominal.
Scénario alternatif SA3	SA3 commence au point 11 du scénario nominale. 12. L'utilisateur refuse le ticket. Puis 13 et 14 du scénario nominal. 12. Le DAB délivre les billets. 13. L'utilisateur prend les billets.
Scénario d'exception SE1 : Carte non valide	SE1 commence au point 2 du scénario nominal. Le DAB Indique que la carte n'est pas valide restitue la carte et met fin au cas.
Scénario d'exception SE2 : Le code est erroné pour la troisième fois	SE2 commence au point 5 du scénario nominal. Le DAB Indique que le code est erroné pour la troisième fois, confisque la carte et met fin au cas.
Scénario d'exception SE3 : Retrait non autorisé	SE3 commence au point 6 du scénario nominal. Le DAB Indique que tout retrait est impossible, restitue la carte et met fin au cas.

UML – Description textuelle des cas d'utilisation

❑ Exemple de description textuelle : Le cas d'utilisation **‘Retirer de l’argent’**

Scénario d’exception SE4 : Carte non reprise	SE4 commence au point 11 du scénario nominal. Au bout de 30s, le DAB confisque la carte et met fin au cas.
Scénario d’exception SE5 : Billets non pris	SE5 commence au point 15 du scénario nominal. Au bout de 30s, le DAB reprend les billets et met fin au cas.
Scénario d’exception SE6 : Annulation de la transaction	SE6 peut démarrer entre les points 4 et 9 du scénario nominal. Le DAB éjecte la carte et met fin au cas.
Post-conditions	Les détails de la transaction doivent être enregistrés (montant, numéro carte, date...) aussi bien en cas de succès que d’échec.
Exigences non fonctionnelles	Sécurité : La saisie du code confidentiel ne doit pas faire apparaître le code à l’écran.

UML – Diagramme des cas d'utilisation

❏ Exercice d'application

Une bibliothèque municipale souhaite informatiser la gestion de l'emprunt de livres.

Un usager peut :

- Consulter le catalogue en ligne.
- S'inscrire comme membre.
- Emprunter un livre s'il est membre et que le livre est disponible.
- Retourner un livre.

Un bibliothécaire peut :

- Enregistrer l'inscription d'un nouvel usager.
 - Enregistrer les emprunts et les retours de livres.
 - Mettre à jour le catalogue (ajouter/supprimer des livres).
-
- **Choisir les 2 cas d'utilisation les plus importants de ce système.**
 - **Réaliser le tableau de description textuelle de ces 2 cas d'utilisation.**

UML – Diagramme des cas d'utilisation

❏ Exercice d'application

Dans une entreprise X, le traitement des commandes clients et de la facturation est le suivant :

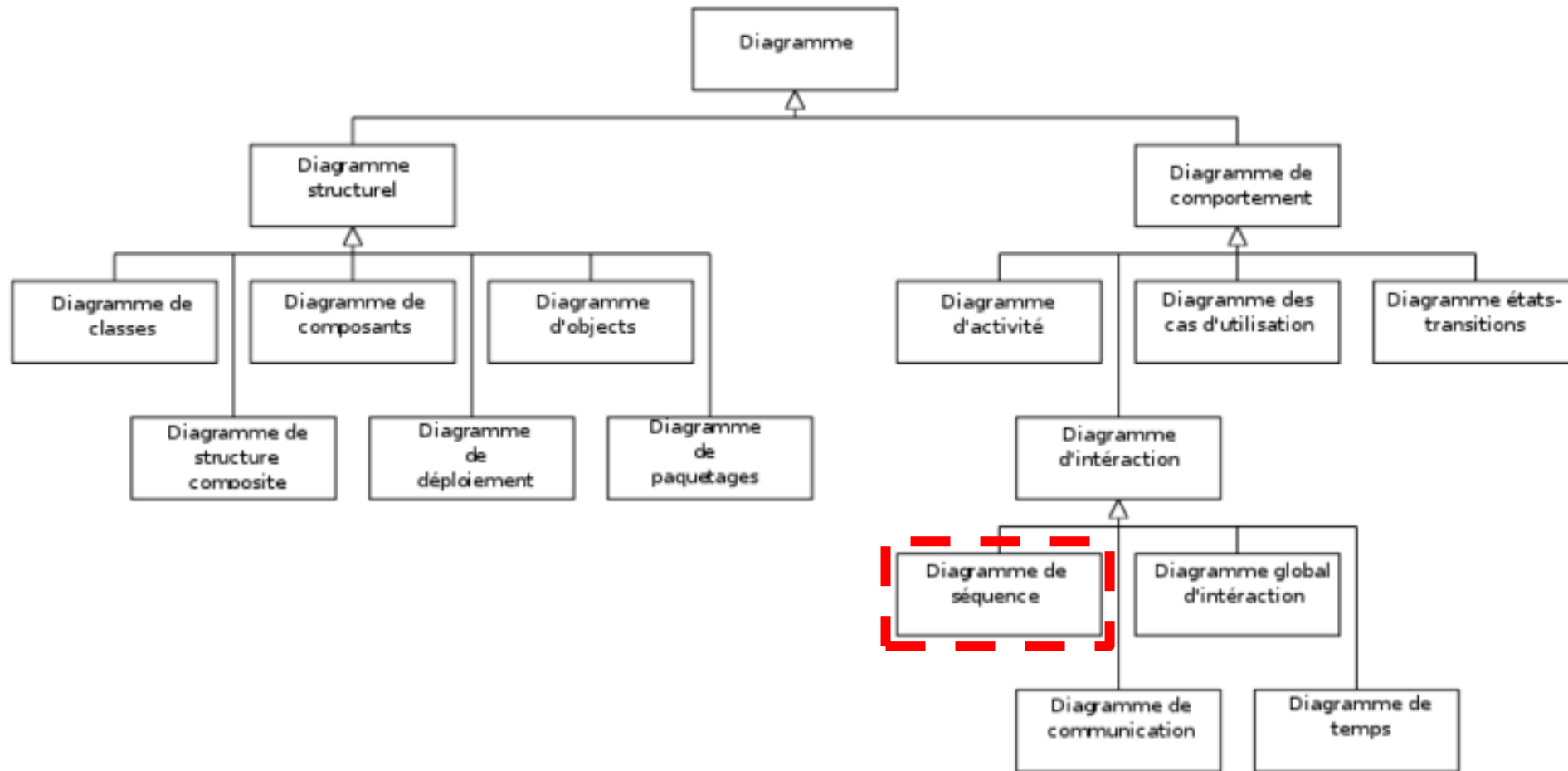
Les commandes clients arrivent chez la secrétaire au matin. Celle-ci la remet en début d'après-midi au service préparation des livraisons. Dès que la commande parvient au responsable, celui-ci vérifie l'identité du client et examine les stocks, si les stocks sont suffisants il rédige un bon de préparation sinon il adresse un courrier type au client et la commande est mise en attente. Lorsqu'un préparateur est disponible, celui-ci prépare la livraison à l'aide du bon de préparation : il prélève les marchandises, les emballe, saisit les bons de préparation et édite en double exemplaire le bon de livraison dont un exemplaire est adressé au client en même temps que les colis, le deuxième exemplaire est transmis au service comptable.

Le service comptable établit les factures : A partir du bon de livraison, l'opératrice comptable saisit le n° du bon, vérifie les tarifs et les conditions de règlement et édite la facture en double exemplaire : un exemplaire est adressé au client, l'autre est archivé en attente de comptabilisation

L'enregistrement comptable des factures : l'opératrice comptable saisit le n° de facture et valide les données à l'écran. Après saisie, le grand livre est mis à jour.

- **Choisir les 2 cas d'utilisation les plus importants de ce système.**
- **Réaliser le tableau de description textuelle de ces 2 cas d'utilisation.**

UML – Diagramme de Séquence

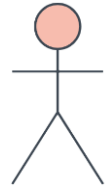


UML – Diagramme de Séquence

- ❑ Il permet de représenter des échanges entre les différents objets et acteurs du système **en fonction du temps**.
- ❑ Les messages sont échangés entre les lignes de vie, présentés dans un ordre chronologique (la dimension verticale).
- ❑ A moins que le système à modéliser soit extrêmement simple, nous ne pouvons pas modéliser la dynamique globale du système dans un seul diagramme.
- ❑ Solution : faire appel à un ensemble de diagrammes de séquences chacun illustre un cas d'utilisation.
- ❑ Il existe deux types de diagrammes de séquence : **Diagramme de séquence analyse** (boite noire) **et diagramme de séquence conception** (boite blanche).
- ❑ Composants : ligne de vie, messages asynchrones, messages synchrones, messages de création / suppression d'une instance, événement, fragment d'interaction combiné.

UML – Diagramme de Séquence

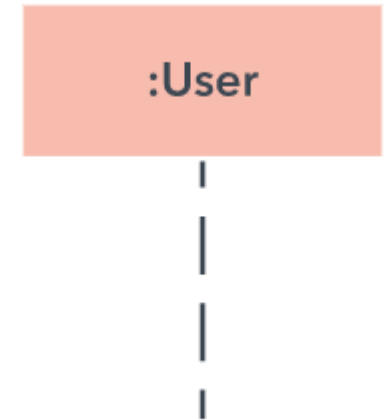
- ❑ **Acteur** : les entités qui interagissent avec le système ou qui sont extérieures à lui.
- ❑ **Boîte d'activation** : Représente le temps nécessaire pour qu'un objet accomplisse une tâche. Plus la tâche nécessite de temps, plus la boîte d'activation est longue.



UML – Diagramme de Séquence

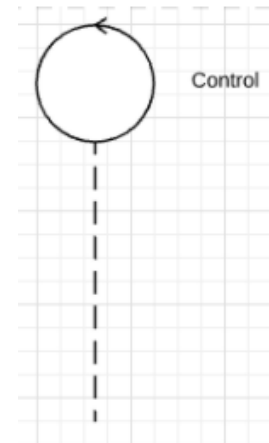
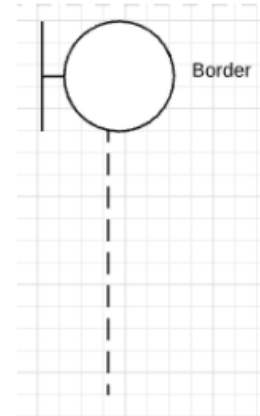
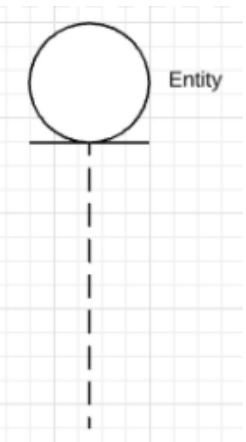
❑ Ligne de vie :

- Se représente par un rectangle, auquel est accroché une ligne verticale pointillée, contenant une étiquette dont la syntaxe est : **objet : classe**.
- Au moins un des deux noms doit être spécifié dans l'étiquette, les deux points (:) sont, quant à eux, obligatoires.
- Représente le passage du temps qui se prolonge vers le bas.
- Cette ligne verticale en pointillés montre les événements séquentiels affectant un objet au cours du processus schématisé.
- Les lignes de vie peuvent commencer par une forme rectangulaire avec un intitulé ou par un symbole d'acteur.



UML – Diagramme de Séquence

- ❑ **Frontière** : des interfaces qui interagissent avec des acteurs externes. Ces objets peuvent, par exemple, être des interfaces utilisateur où l'acteur serait alors une personne.
- ❑ **Entité** : représentent des conteneurs de données ou des objets qui contiennent des données système.
- ❑ **Contrôle** : Pour que les frontières et les entités communiquent, il faut faire appel à un élément de contrôle. L'élément de contrôle relie l'entité et la frontière en tant que médiateur. Il surveille les signaux des deux éléments et vérifie leur logique.



UML – Diagramme de Séquence

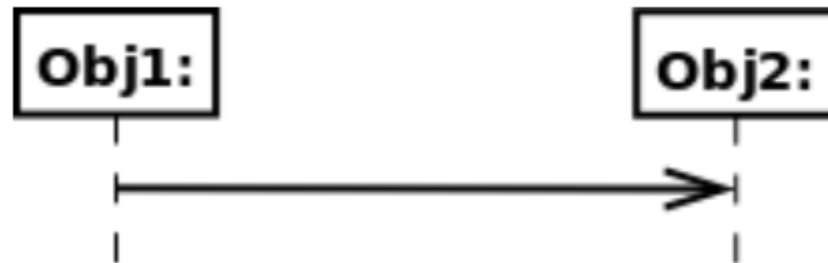
❑ Message :

- ❑ Un message définit une communication particulière entre des lignes de vie.
- ❑ Plusieurs types de messages existent, les plus communs sont :
 - l'envoi d'un signal ;
 - l'invocation d'une opération ;
 - la création ou la destruction d'une instance.

UML – Diagramme de Séquence

❑ Message asynchrone :

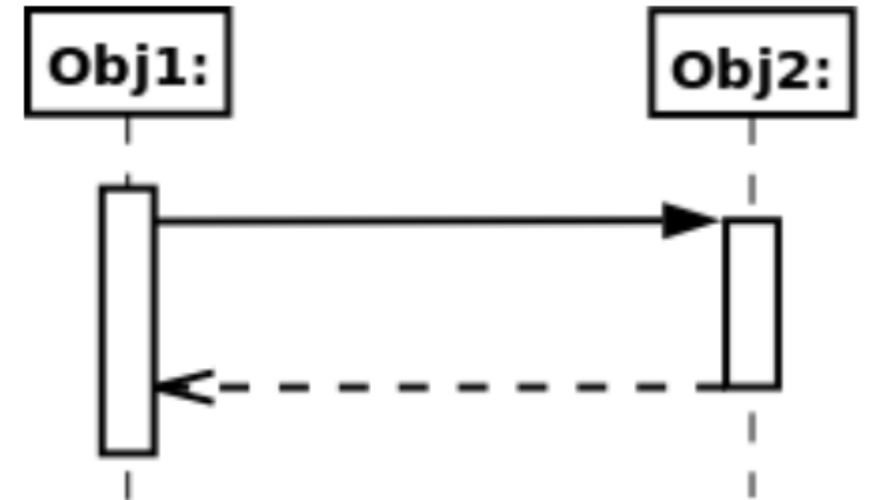
- ❑ Une interruption ou un événement sont des exemples de signaux : Ils n'attendent pas de réponse et ne bloquent pas l'émetteur qui ne sait pas si le message arrivera à destination, le cas échéant quand il arrivera et s'il sera traité par le destinataire.
- ❑ Un signal est, par définition, un message asynchrone.
- ❑ Graphiquement, un message asynchrone se représente par une flèche en traits pleins et à l'extrémité ouverte partant de la ligne de vie d'un objet expéditeur et allant vers celle de l'objet cible.



UML – Diagramme de Séquence

❑ Message synchrone :

- ❑ La plupart des invocations sont synchrones, l'émetteur reste bloqué le temps que dure l'invocation de l'opération.
- ❑ Graphiquement, un message synchrone se représente par une flèche en traits pleins et à l'extrémité pleine partant de la ligne de vie d'un objet expéditeur et allant vers celle de l'objet cible.
- ❑ Ce message peut être suivi d'une réponse qui se représente par une flèche en pointillé.



UML – Diagramme de Séquence

❑ Message reflexive :

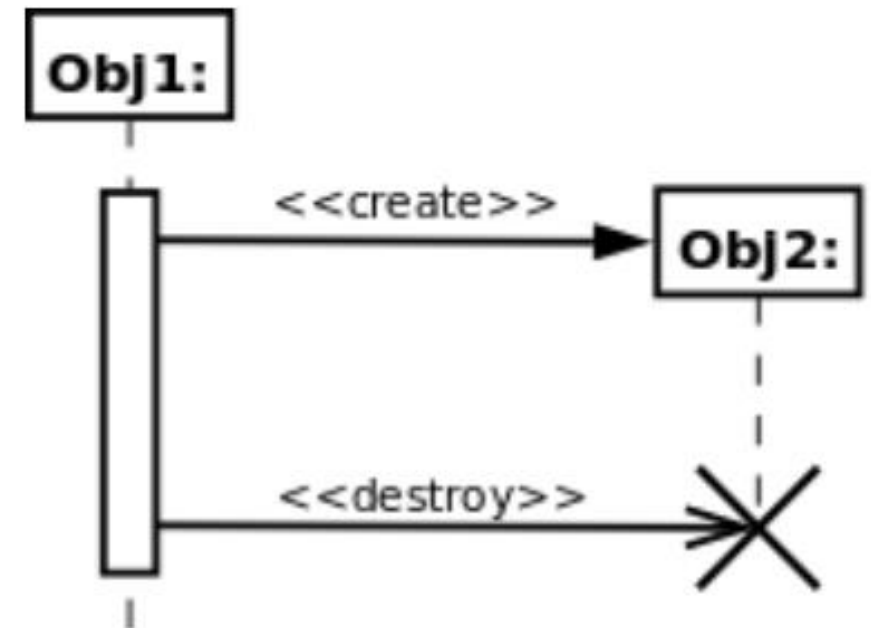
- ❑ L'objets envoie un message à lui-même.
- ❑ L'expéditeur est lui-même le destinataire.



UML – Diagramme de Séquence

❑ Messages de création et destruction d'instance :

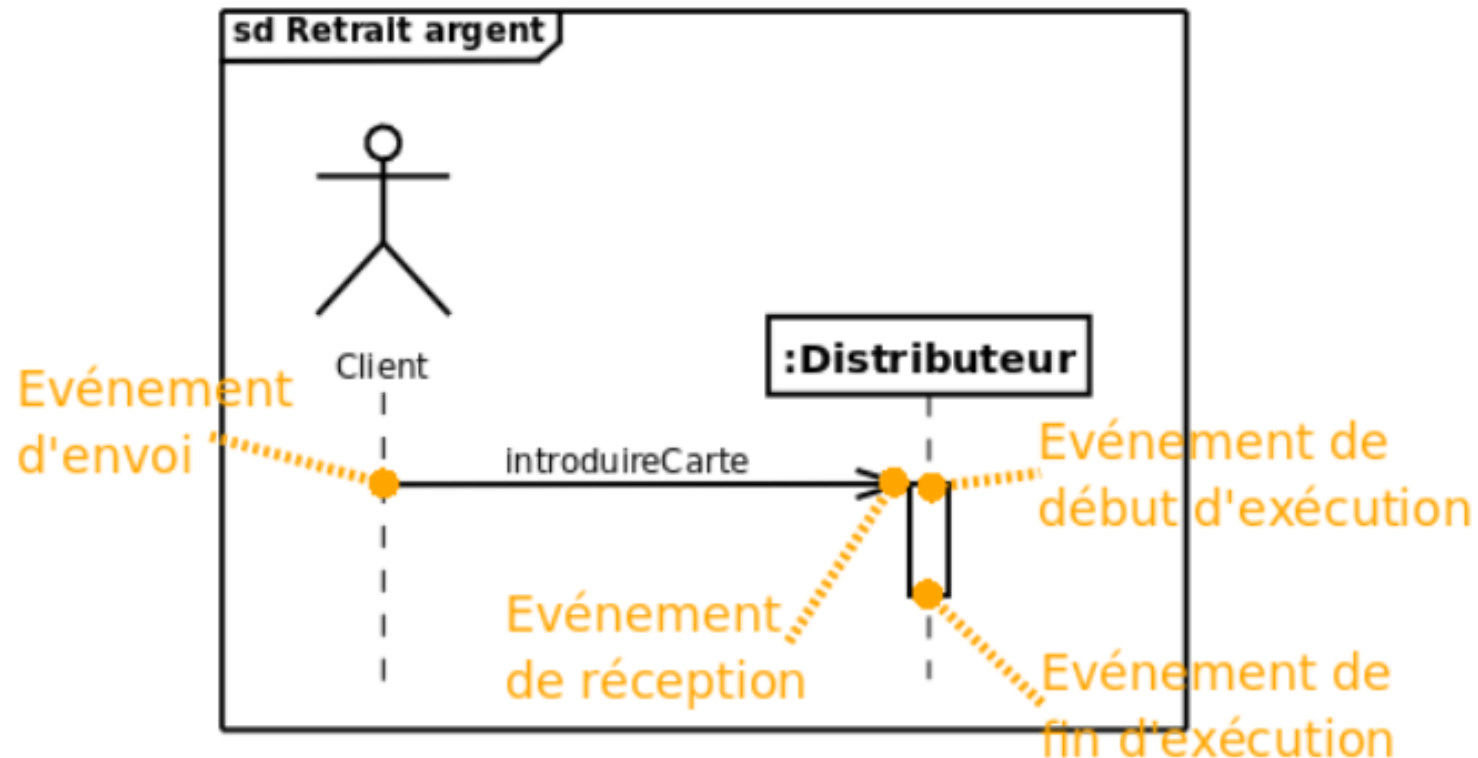
- ❑ La création d'un objet est matérialisée par une flèche qui pointe sur le sommet d'une ligne de vie.
- ❑ La destruction d'un objet est matérialisée par une croix qui marque la fin de la ligne de vie de l'objet. La destruction d'un objet n'est pas nécessairement consécutive à la réception d'un message.



UML – Diagramme de Séquence

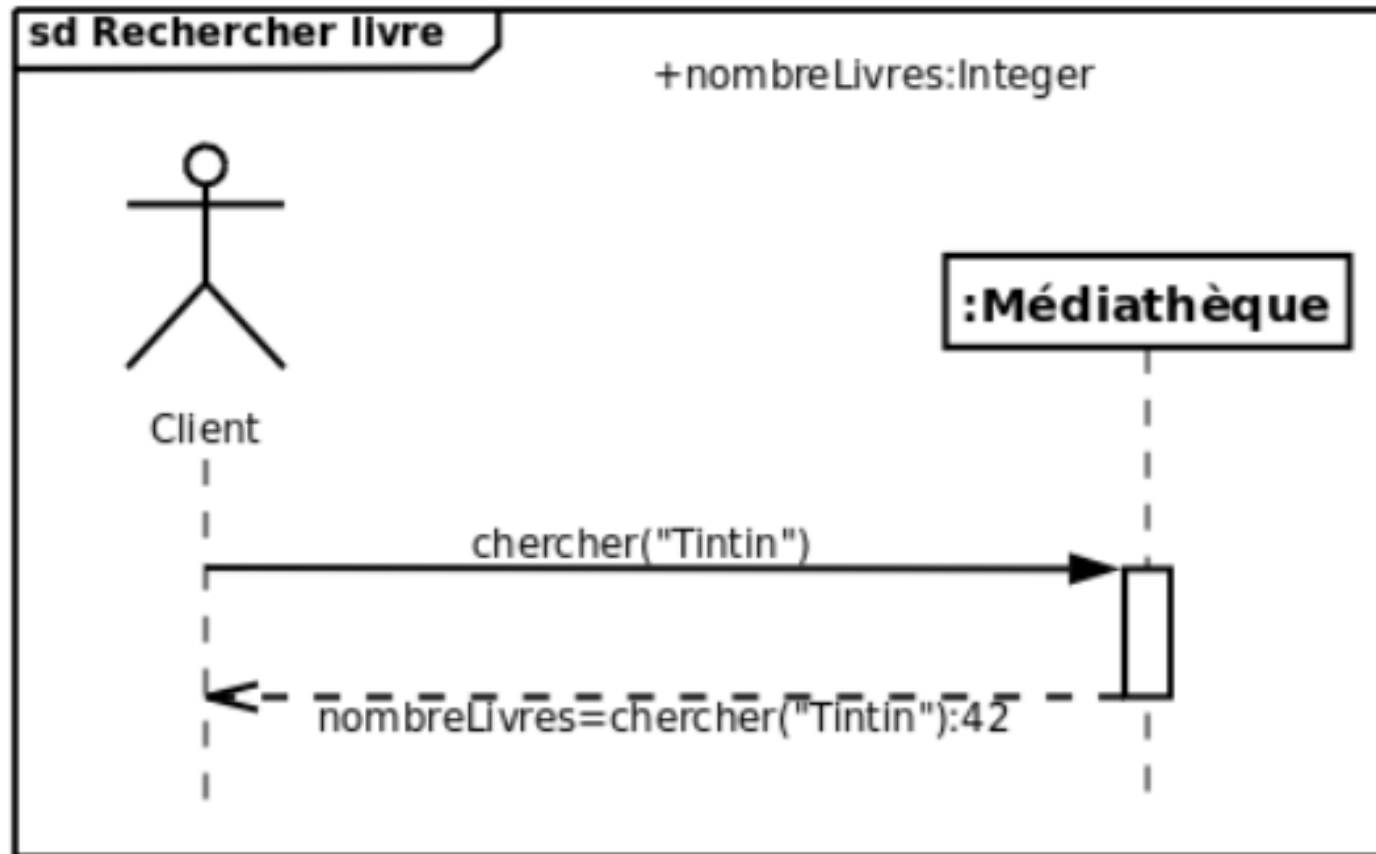
□ Événements et messages :

- UML permet de séparer clairement l'envoi du message, sa réception, ainsi que le début de l'exécution de la réaction et sa fin.



UML – Diagramme de Séquence

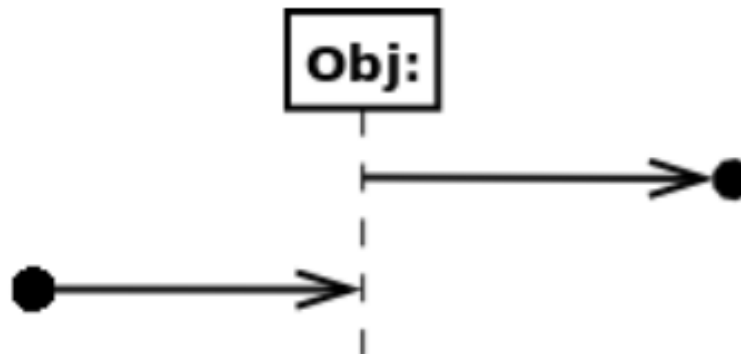
❑ Événements et messages :



UML – Diagramme de Séquence

❑ Message perdu et trouvé :

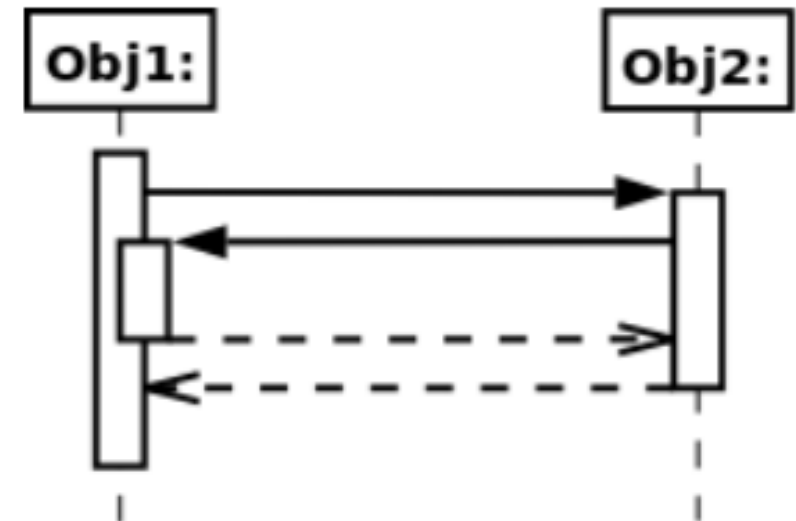
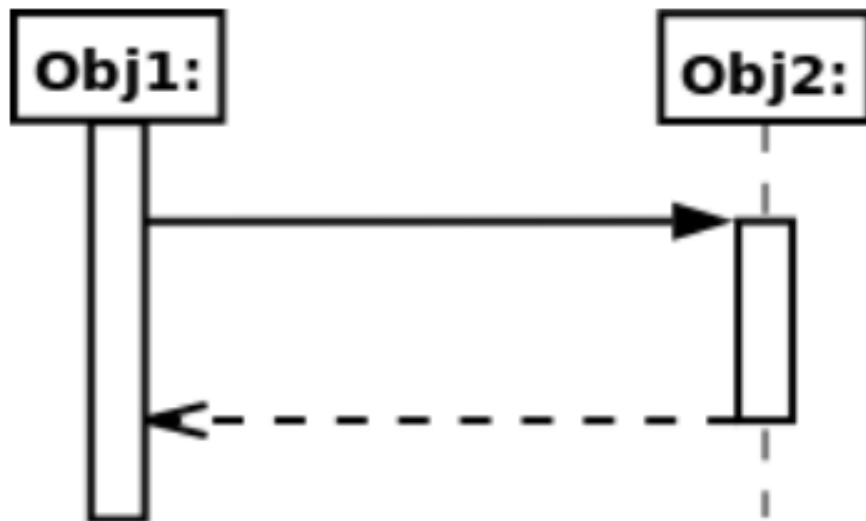
- ❑ Un message complet est tel que les événements d'envoi et de réception sont connus. Un message complet se représente par une simple flèche dirigée de l'émetteur vers le récepteur.
- ❑ Un message perdu est tel que l'événement d'envoi est connu, mais pas l'événement de réception. Il se représente par une flèche qui pointe sur une petite boule noire.
- ❑ Un message trouvé est tel que l'événement de réception est connu, mais pas l'événement d'émission. Une flèche partant d'une petite boule noire représente un message trouvé.



UML – Diagramme de Séquence

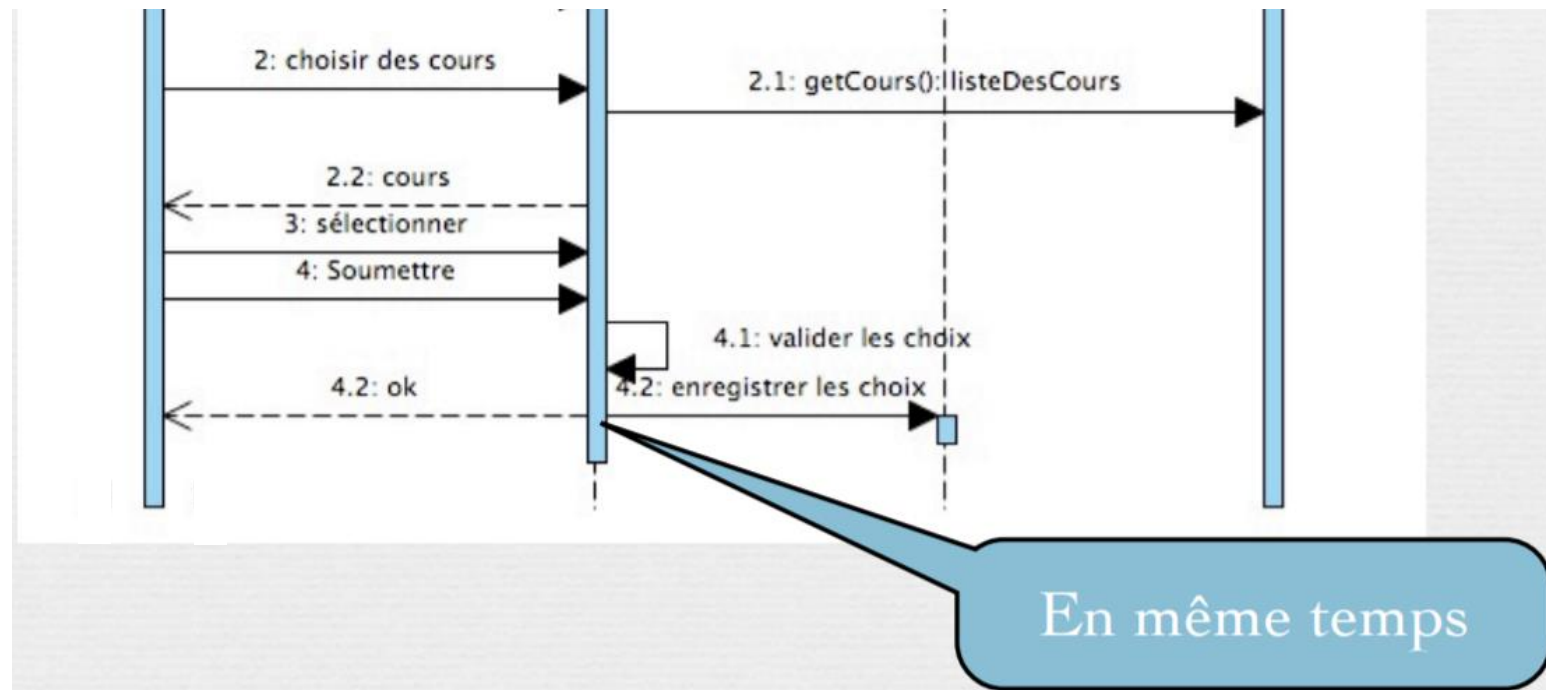
❑ Exécution de méthode et objet actif :

- ❑ Un objet actif initie et contrôle le flux d'activités. Graphiquement, la ligne pointillée verticale d'un objet actif est remplacée par un double trait vertical.
- ❑ Un objet passif, au contraire, a besoin qu'on lui donne le flux d'activité pour pouvoir exécuter une méthode. La spécification de l'exécution d'une réaction sur un objet passif se représente par un rectangle blanc ou gris placé sur la ligne de vie en pointillé.



UML – Diagramme de Séquence

❑ Messages :

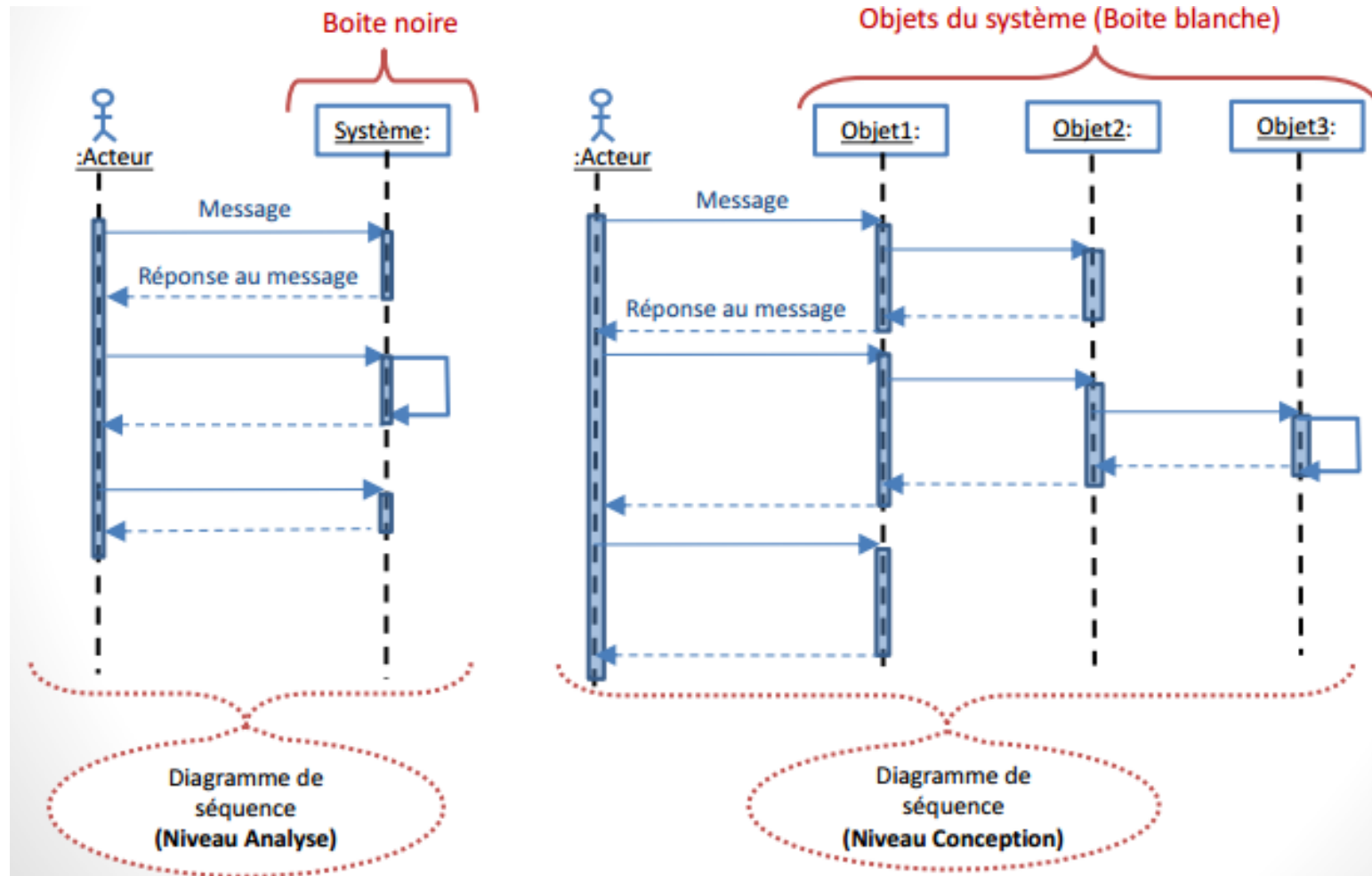


UML – Diagramme de Séquence

- ❑ Exo 1 : Réaliser le diagramme de séquence d'analyse du cas d'utilisation « Publier message sur Forum Y ».
- ❑ Exo 2 : Réaliser le diagramme de séquence de conception de « Changer prix article » par un administrateur de la plateforme.
- ❑ Exo 3 : Réaliser le diagramme de séquence de conception de « S'authentifier via compte Facebook ».
- ❑ Exo 4 : Réaliser le diagramme de séquence de conception de « Exécuter une requête ».
- ❑ Exo 5 : Réaliser le diagramme de séquence de conception de « S'inscrire à une formation » entant qu'étudiant dans une université.
- ❑ Exo 6 : Réaliser le diagramme de séquence d'analyse de « Contacter administrateur » tel que traiter par l'administrateur.

UML – Diagramme de Séquence

- Diagramme de Séquence d'Analyse (boite noire) vs de Conception (boite blanche) :

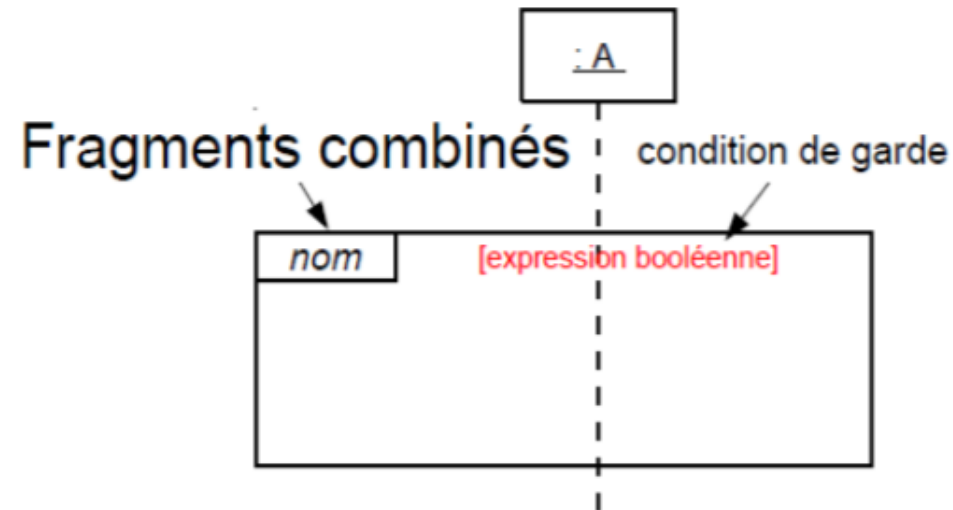


UML – Diagramme de Séquence

❑ Fragments d'interaction combinés :

- ❑ Représente des articulations d'interactions.
- ❑ Les fragments combinés permettent de décrire des diagrammes de séquence de manière compacte.
- ❑ Il existe 13 Fragments combinés.

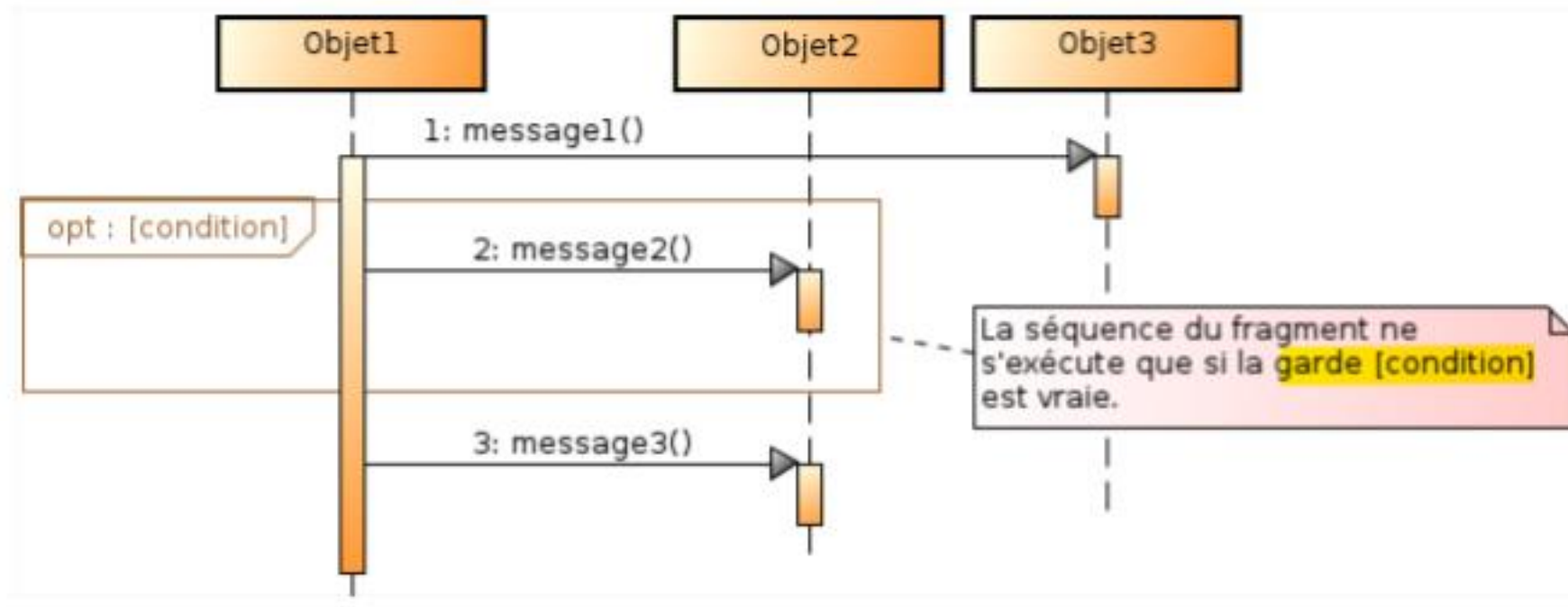
1. Opt
2. Loop
3. Alt
4. Break
5. Critique
6. Ref
7. ...



UML – Diagramme de Séquence

❑ Fragments d'interaction combinés :

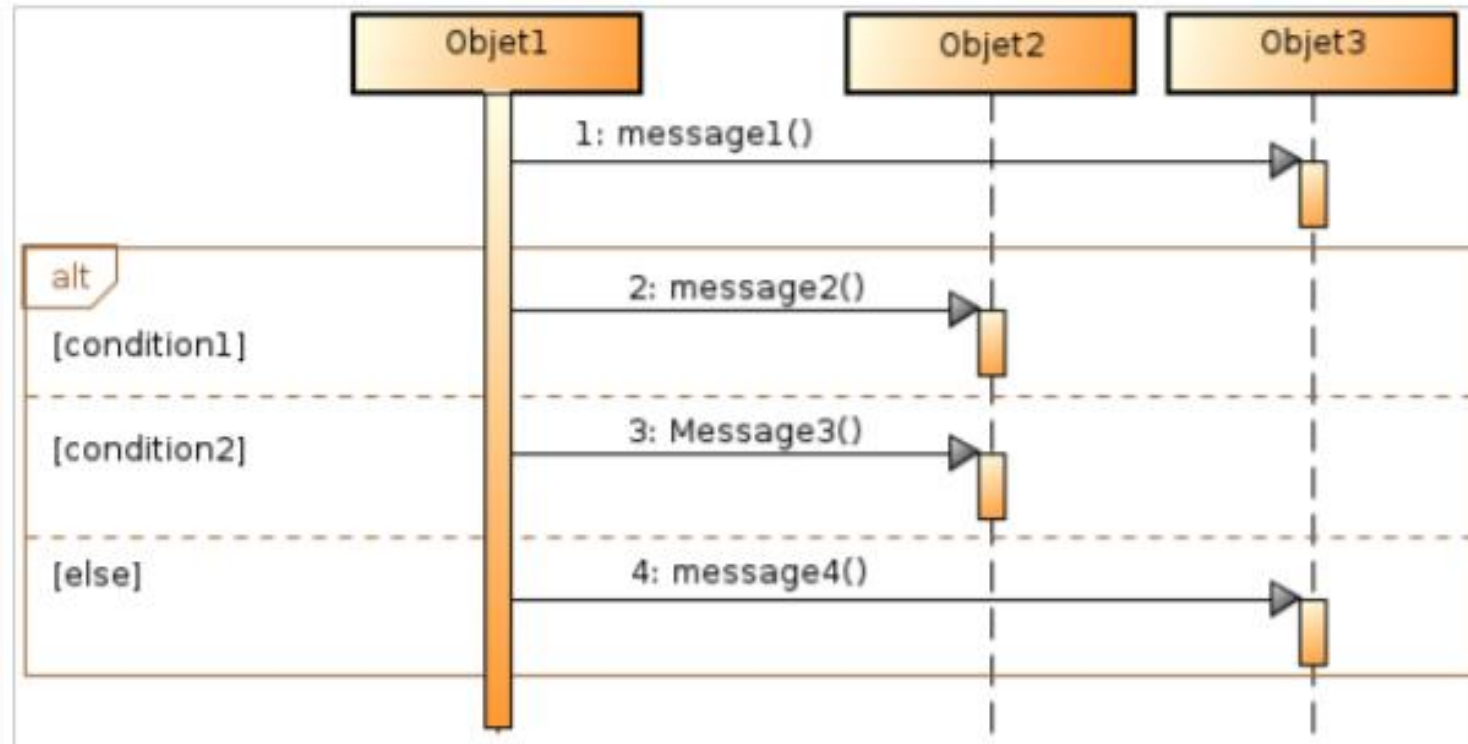
- ❑ **Opt** : Fragment parcouru si une condition est vérifiée



UML – Diagramme de Séquence

❑ Fragments d'interaction combinés :

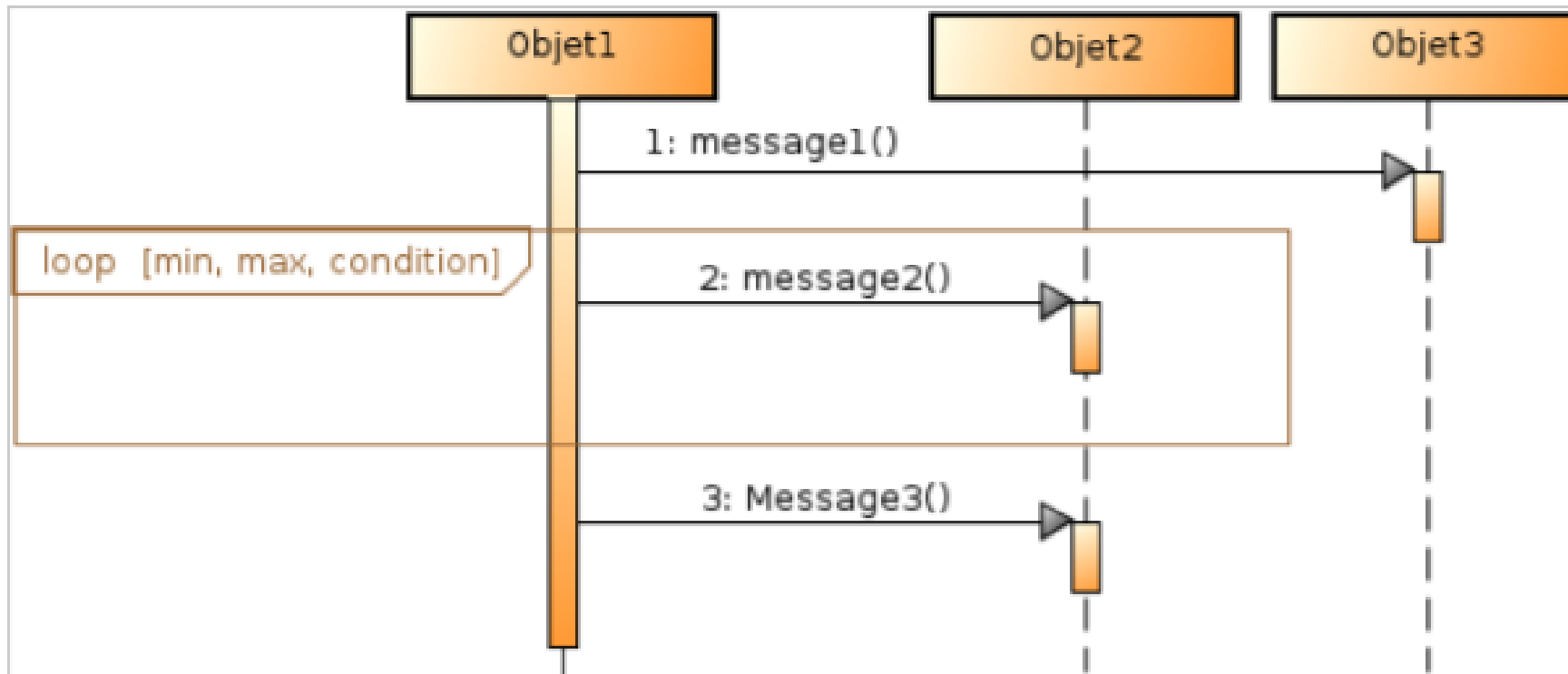
❑ **Alt** : Equivalent à la structure de contrôle "si .. alors .. sinon"



UML – Diagramme de Séquence

❑ Fragments d'interaction combinés :

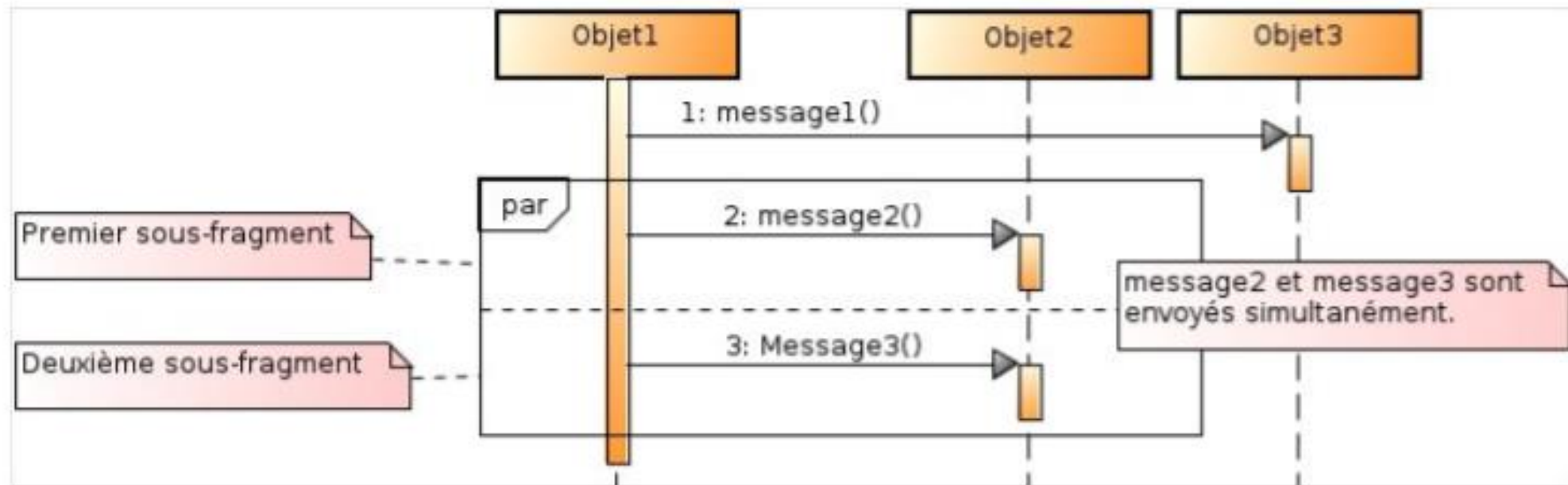
❑ **Loop** : Répétition du fragment tant que la condition est vérifiée



UML – Diagramme de Séquence

❑ Fragments d'interaction combinés :

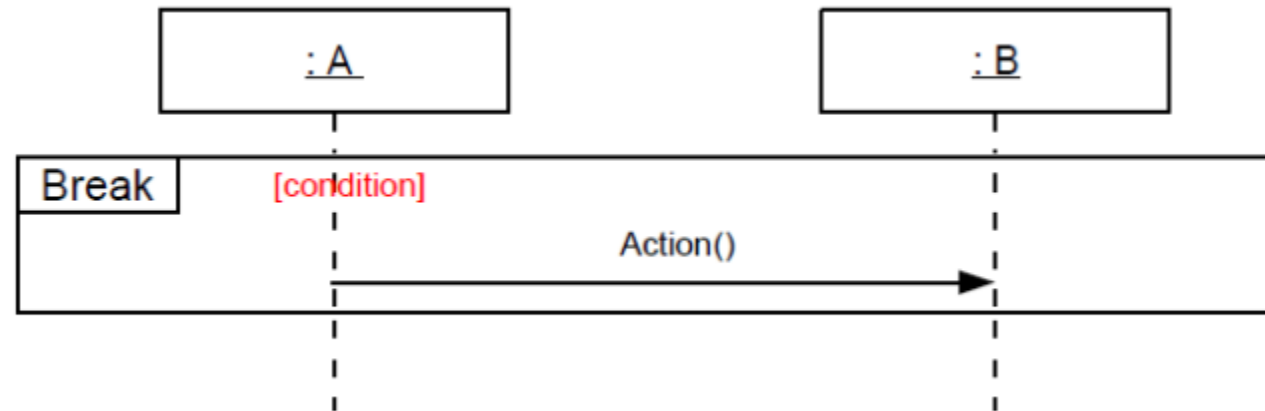
- ❑ **Par** : Un fragment d'interaction avec l'opérateur de traitements parallèles contient au moins deux sous fragments (opérandes) séparés par des pointillés qui s'exécutent simultanément (traitements concurrents)



UML – Diagramme de Séquence

❑ Fragments d'interaction combinés :

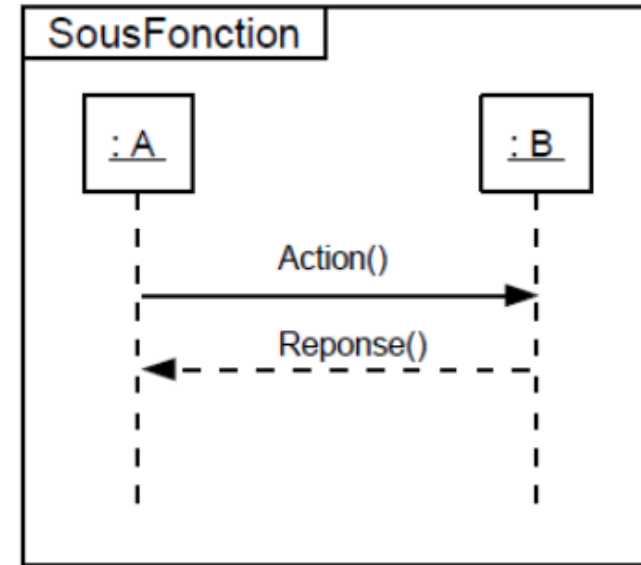
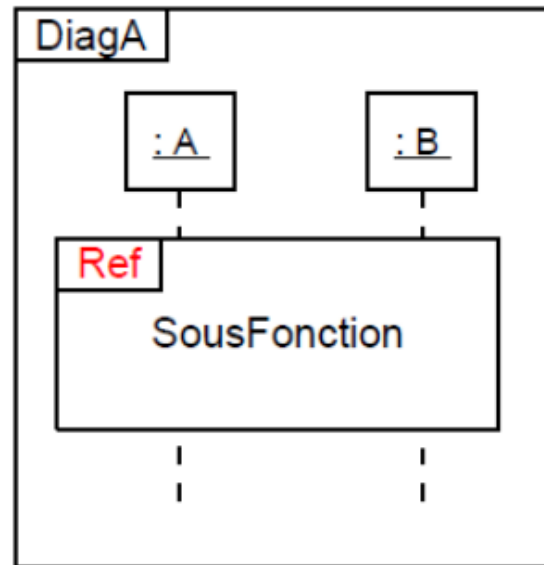
- ❑ **Break** : pour les fragments représentant des scénarios exceptionnels ou de ruptures (ex appui sur la touche « Esc »). Le scénario de rupture est exécuté si une condition de garde est satisfaite.



UML – Diagramme de Séquence

❑ Fragments d'interaction combinés :

- ❑ **Ref** : permet de faire appel à un autre diagramme de séquence.
- ❑ Les inclusions et les extensions sont des cas typiques d'utilisation de la réutilisation par référencement



UML – Diagramme de Séquence

❑ Exercice d'application

Une bibliothèque municipale souhaite informatiser la gestion de l'emprunt de livres.

Un usager peut :

- Consulter le catalogue en ligne.
- S'inscrire comme membre.
- Emprunter un livre s'il est membre et que le livre est disponible.
- Retourner un livre.

Un bibliothécaire peut :

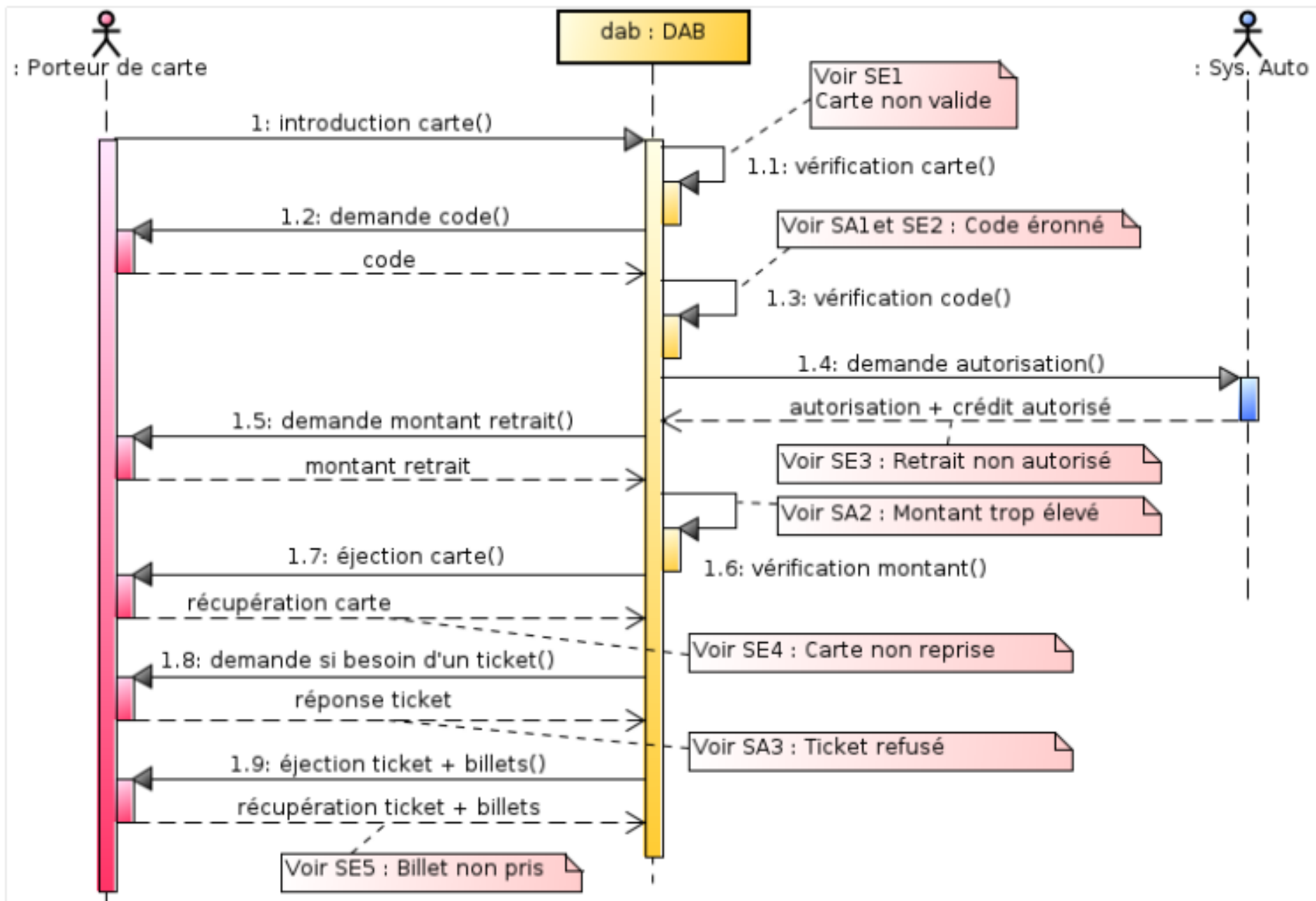
- Enregistrer l'inscription d'un nouvel usager.
 - Enregistrer les emprunts et les retours de livres.
 - Mettre à jour le catalogue (ajouter/supprimer des livres).
-
- **Choisir les 2 cas d'utilisation les plus importants de ce système.**
 - **Réaliser le diagramme de séquence de ces 2 cas d'utilisation.**

UML – Description textuelle des cas d'utilisation

□ Donner le Diagramme de Séquence du cas d'utilisation **‘Retirer de l’argent’**

Scénario nominal

1. Le Porteur de carte introduit sa carte dans le DAB.
2. Le DAB vérifie que la carte introduite est bien une carte bancaire.
3. Le DAB demande le code de la carte au Porteur de carte.
4. Le Porteur de carte saisit son code.
5. Le DAB compare ce code avec celui qui est codé sur la carte.
6. Le DAB demande une autorisation au Système Globale d'autorisation.
7. Le Système d'Autorisation globale donne son accord et indique le crédit hebdomadaire.
8. Le DAB demande le montant désiré au Porteur de carte.
9. Le Porteur de carte saisit le montant.
10. Le DAB vérifie si le montant demandé est inférieur ou égale au crédit hebdomadaire.
11. Le DAB demande au Porteur de carte s'il désire un ticket.
12. Le Porteur de carte accepte le ticket.
13. Le DAB rend la carte et demande au Porteur de carte de la retirer.
14. Le Porteur de carte reprend sa carte.
15. Le DAB délivre le ticket et les billets.
16. Le Porteur de carte prend les billets et le ticket.



UML – Diagramme d'activité

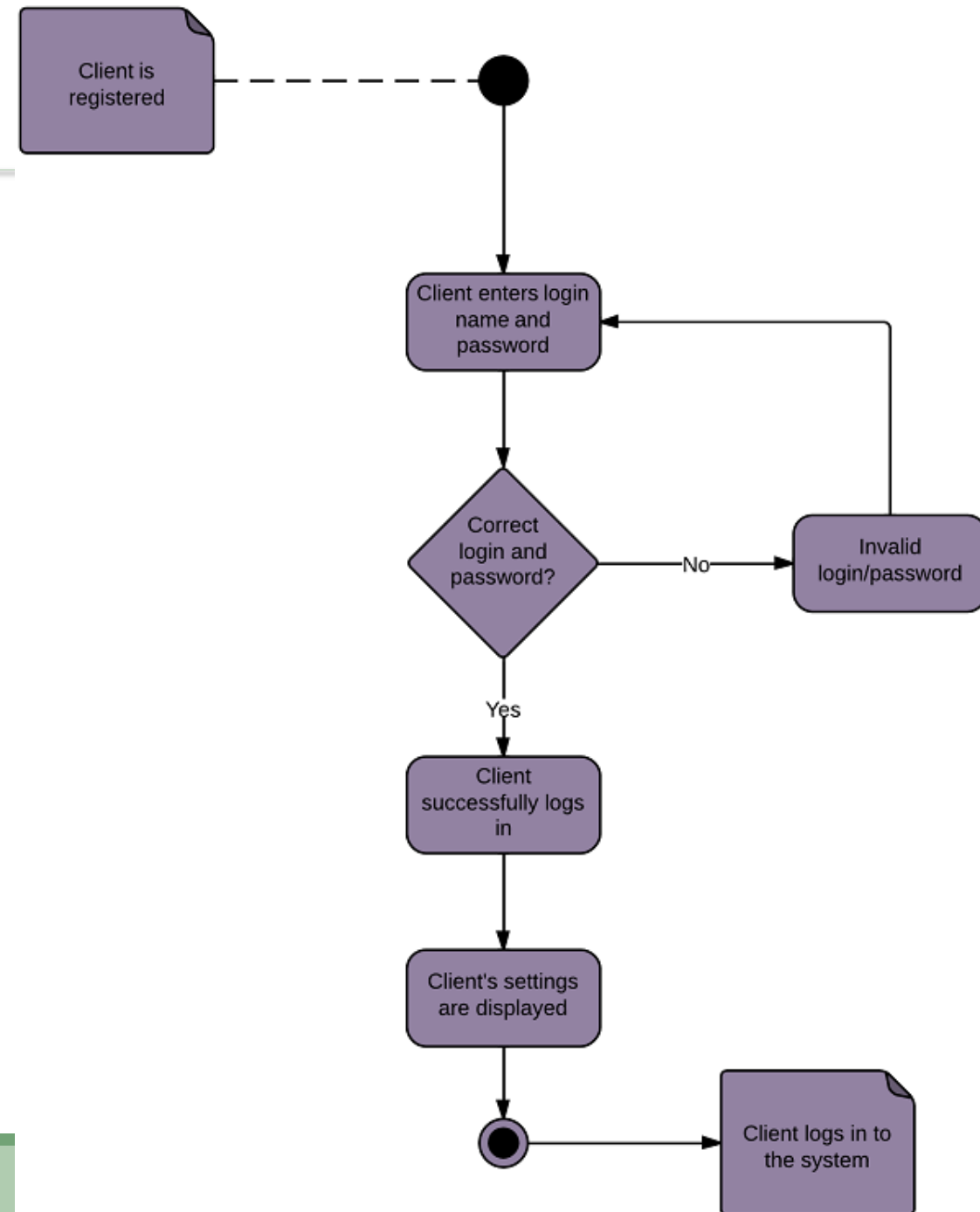
- ❑ Dans UML, un diagramme d'Activités est utilisé pour afficher la séquence d'activités.
- ❑ Les diagrammes d'activité affichent le flux de travail d'un point de départ à un point d'arrivée en détaillant les nombreux chemins de décision existant dans la progression des événements contenus dans l'activité.
- ❑ Ils peuvent être utilisés pour détailler des situations dans lesquelles un traitement parallèle peut avoir lieu lors de l'exécution de certaines activités.
- ❑ Les diagrammes d'activité sont utiles pour la modélisation métier car ils sont utilisés pour détailler les processus impliqués dans les activités métier.
- ❑ Un diagramme d'activités visualise un graphe d'activités qui modélise le comportement interne d'une méthode (une réalisation d'une opération), d'un cas d'utilisation ou plus généralement d'un processus impliquant un ou plusieurs classificateurs (classes / cas d'utilisation / paquetages /...).

UML – Diagramme d'activité

- ❑ Un diagramme d'activités représente l'état d'exécution d'un mécanisme, sous la forme d'un déroulement d'étapes regroupées séquentiellement dans des branches parallèles de flots de contrôle.
- ❑ Il est utile pour la représentation des processus métiers et les cas d'utilisation.
- ❑ Le diagramme d'activités comprend :
 - **des activités** (une activité = une étape d'exécution, état-activité). Une activité représente une exécution d'un mécanisme, un déroulement d'étapes séquentielles. Le passage d'une activité vers une autre est matérialisé par une transition.
 - **des transitions** qui sont automatiques entre activités, il est inutile également de préciser les événements. Les transitions sont déclenchées par la fin d'une activité et provoquent le début immédiat d'une autre.
- ❑ En théorie, tous les mécanismes dynamiques pourraient être décrits par un diagramme d'activités, mais seuls les mécanismes complexes ou intéressants méritent d'être représentés.

UML – Diagramme d'activité

❑ Exemple de diagramme d'activités :



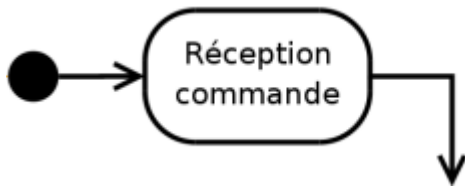
UML – Diagramme d'activité

- ❑ **Activité** : est la spécification d'une séquence paramétrée de comportement. Une activité est représentée comme un rectangle à coins arrondis enfermant toutes les actions, les flux de contrôle et d'autres éléments qui composent l'activité.



UML – Diagramme d'activité

- ❑ **Symbole de début :** (aussi appelé nœud initial) Représente le début d'un processus ou d'un flux de travail dans un diagramme d'activités. Il peut être utilisé seul ou avec un symbole de note qui explique le point de départ.



UML – Diagramme d'activité

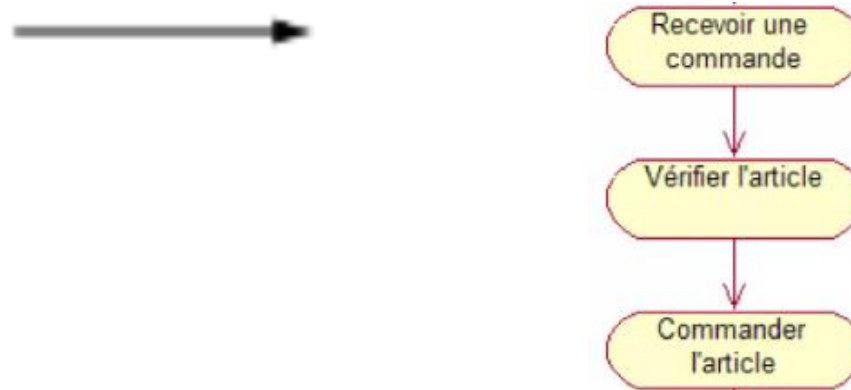
- ❑ **Action** : représente un pas seul dans une activité. Les actions sont dénotées par des rectangles ronds-coincés.



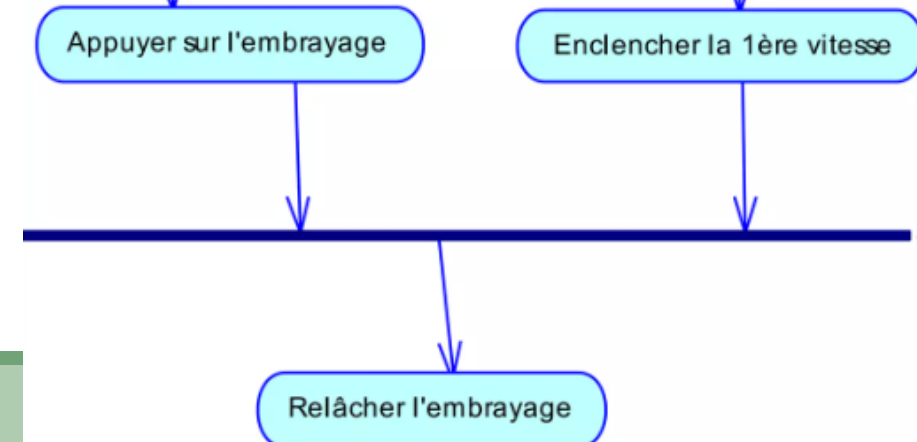
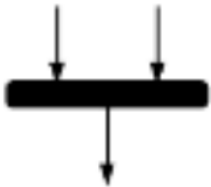
- ❑ Une action est le plus petit traitement qui puisse être exprimé en UML.
- ❑ Une action a une incidence sur l'état du système ou en extrait une information.
- ❑ Les actions sont des étapes discrètes à partir desquelles se construisent les comportements.
- ❑ La notion d'action est à rapprocher de la notion d'instruction élémentaire d'un langage de programmation (comme C++ ou Java).

UML – Diagramme d'activité

- ❑ **Symbole de raccord** : Indique le flux directionnel, ou flux de contrôle, de l'activité. Une flèche entrante marque le début d'une étape d'une activité ; une fois l'étape terminée, le flux se poursuit avec la flèche sortante.

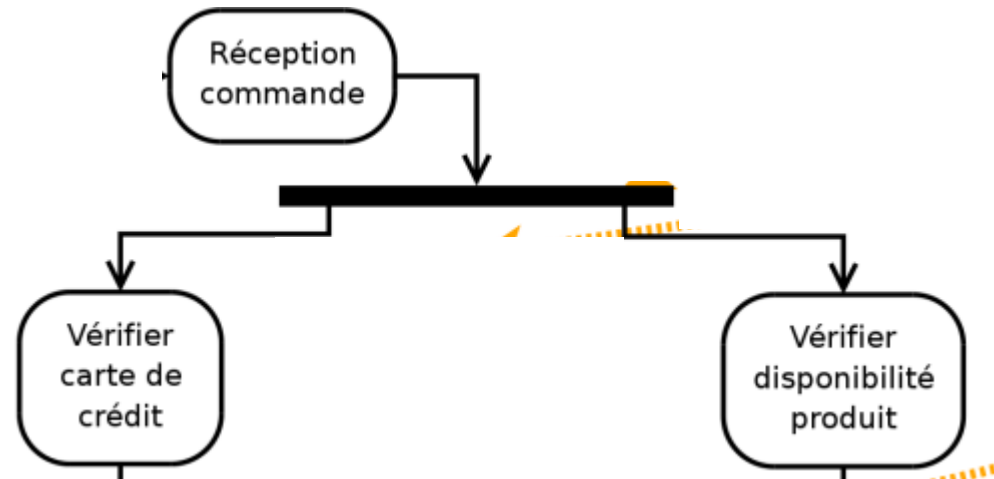
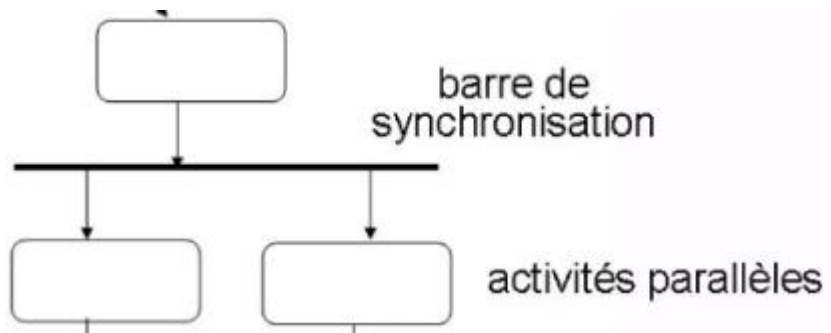


- ❑ **Symbole de raccord/barre de synchronisation** : (aussi appelé nœud d'union/join) Associe deux actions simultanées et les réintroduit dans un flux où n'a lieu qu'une seule action à la fois. Représenté par une ligne verticale ou horizontale épaisse.



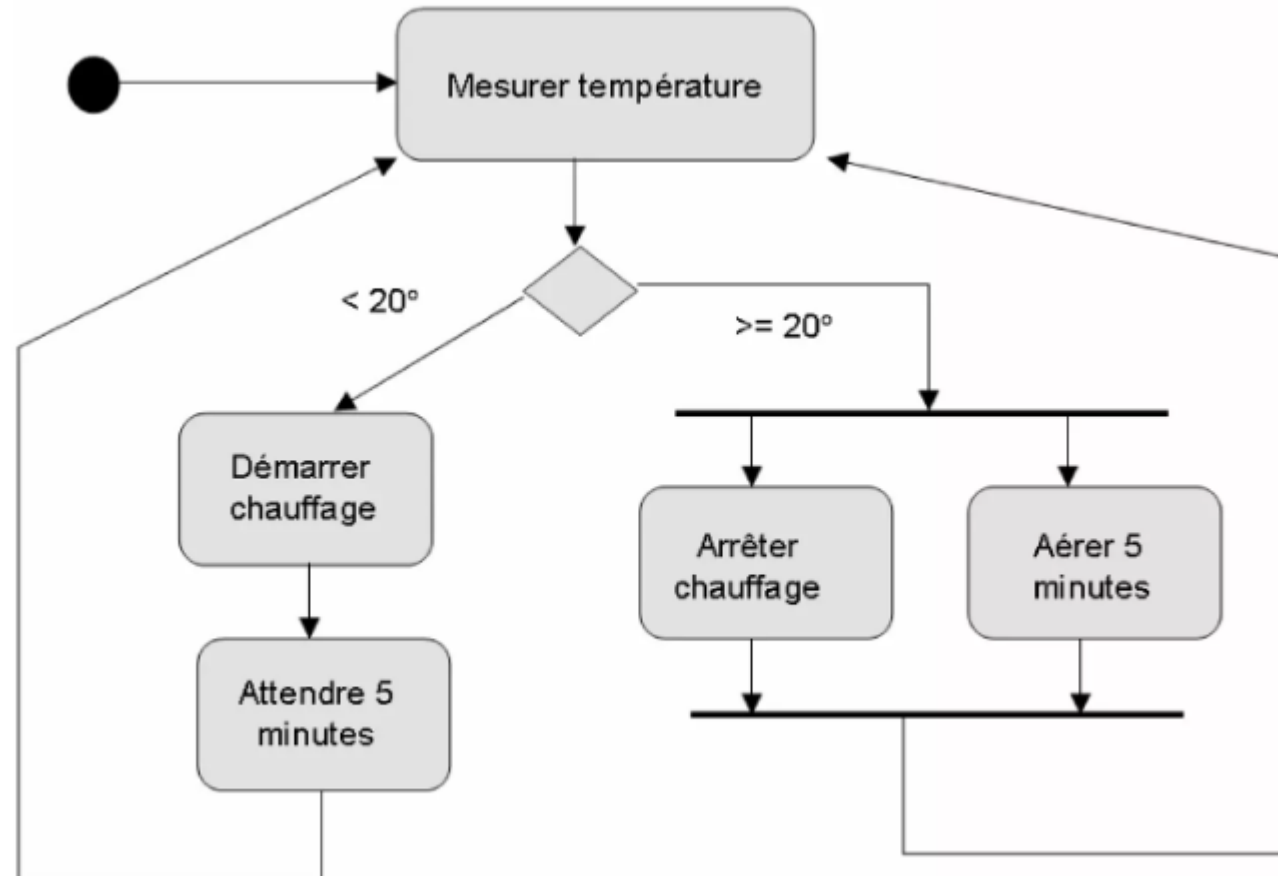
UML – Diagramme d'activité

- ❑ **Symbole d'embranchement : (aussi appelé nœud de bifurcation / fork)** Divise un flux d'activités en deux actions simultanées. Symbolisé par plusieurs lignes fléchées qui partent d'un raccord.



UML – Diagramme d'activité

- ❑ **Symbole de décision** : Représente une décision et possède toujours au moins deux embranchements avec le texte de la condition pour permettre aux utilisateurs de voir les options. Ce symbole représente la ramification ou la fusion de différents flux et sert de cadre ou de conteneur.



UML – Diagramme d'activité

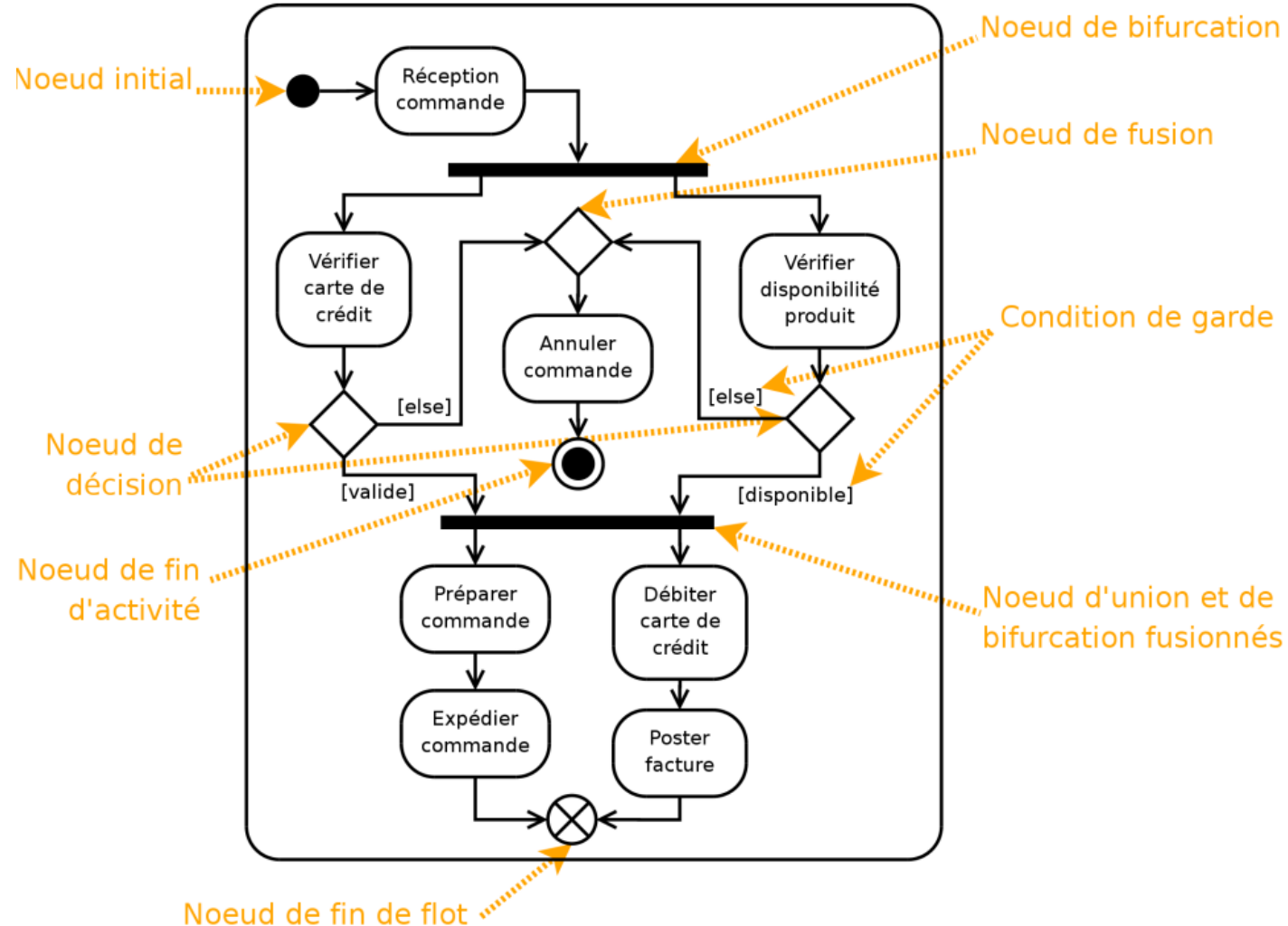
- ❑ **Symbole de fin de flux** : (aussi appelé nœud de fin de flot) Représente la fin d'un schéma de procédé spécifique. Ce symbole ne doit pas représenter la fin de tous les flux dans une activité ; dans ce cas, vous devez utiliser le symbole de fin. Le symbole de fin de flux doit être placé à la fin d'un procédé dans un flux d'activités unique. 
- ❑ **Symbole de fin** : (aussi appelé nœud de fin simple) Marque l'état final d'une activité et représente l'achèvement de tous les flux d'un procédé. 
- ❑ La plupart des diagrammes n'ont qu'un seul flux principal, donc un nœud de fin simple suffit.
- ❑ Le nœud "fin de flux" est utile seulement quand il existe plusieurs chemins parallèles (forks/join) ou plusieurs branches alternatives qu'il faut tous interrompre d'un coup.

UML – Diagramme d'activité

- ❑ **Symbole de fin de flux** : (aussi appelé nœud de fin de flot) Représente la fin d'un schéma de procédé spécifique. Ce symbole ne doit pas représenter la fin de tous les flux dans une activité ; dans ce cas, vous devez utiliser le symbole de fin. Le symbole de fin de flux doit être placé à la fin d'un procédé dans un flux d'activités unique.

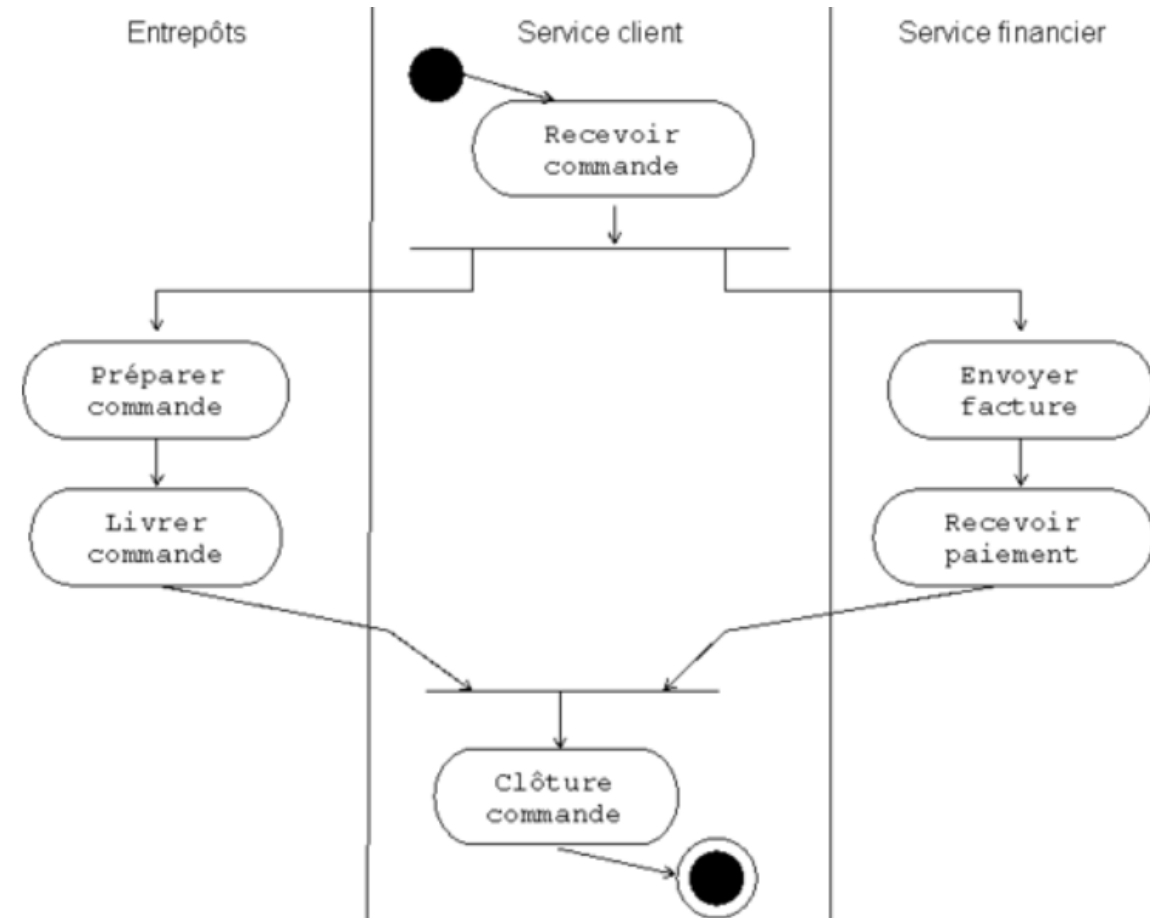


- ❑ **Symbole de fin** : (aussi appelé nœud de fin simple) Marque l'état final d'une activité et représente l'achèvement de tous les flux d'un procédé.



UML – Diagramme d'activité

- ❑ **Couloirs d'activité :**
- ❑ Les diagrammes d'activités indiquent ce qui se passe sans préciser qui fait quoi (en termes de programmation, ils ne précisent pas quelle classe est responsable et en termes de processus métier, ils ne précisent pas quelle partie de l'organisation exécute chaque action).
- ❑ Il est possible de diviser un diagramme d'activités en partitions ou couloirs d'activités (travées, swimlanes).
- ❑ Chaque partition montre quelles actions sont exécutées par une classe ou une unité organisationnelle.



UML – Diagramme d'activité

❑ Exercice d'application

Une bibliothèque municipale souhaite informatiser la gestion de l'emprunt de livres.

Un usager peut :

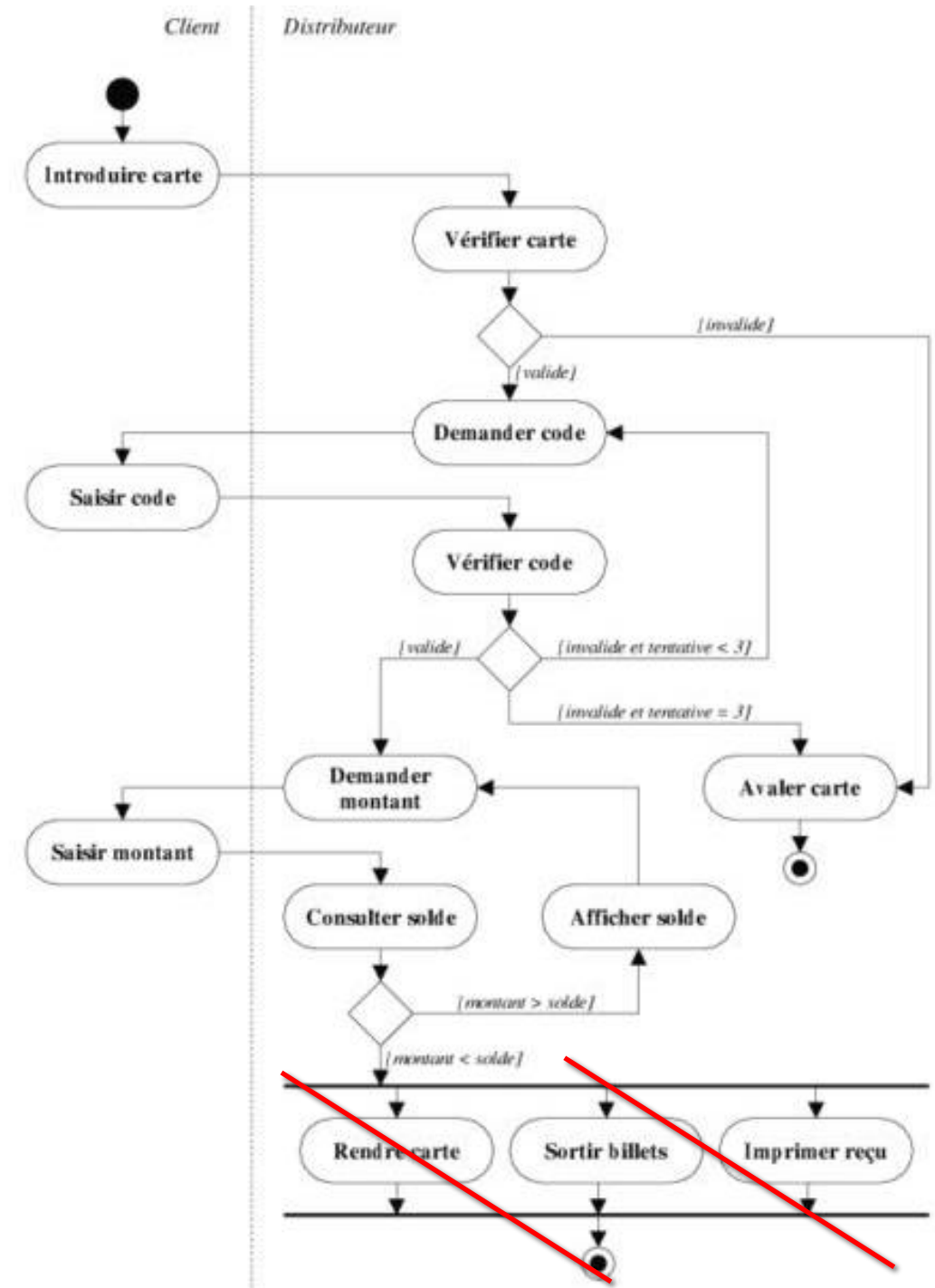
- Consulter le catalogue en ligne.
- S'inscrire comme membre.
- Emprunter un livre s'il est membre et que le livre est disponible.
- Retourner un livre.

Un bibliothécaire peut :

- Enregistrer l'inscription d'un nouvel usager.
 - Enregistrer les emprunts et les retours de livres.
 - Mettre à jour le catalogue (ajouter/supprimer des livres).
-
- **Définir au moins un processus.**
 - **Réaliser le diagramme d'activité relatif à ce processus.**

UML – Diagramme d'activité

□ Diagramme d'activités d'un DAB



UML – Programmation Orientée objet

- ❑ Avant la POO, la programmation procédurale dominait. On écrivait des fonctions qui manipulaient des données globales. Mais à mesure que les logiciels grandissaient, cette approche devenait :
 - difficile à maintenir (une fonction modifiée pouvait casser le reste),
 - difficile à faire évoluer (ajouter une nouvelle fonctionnalité impliquait souvent de tout réécrire),
 - peu réutilisable (le code était spécifique à un contexte).
- ❑ La POO a été inventée pour structurer la complexité : elle permet de penser un logiciel comme un ensemble d'objets interagissant, plutôt qu'une simple suite d'instructions.
- ❑ L'idée fondamentale est de représenter les objets du monde réel dans le programme :
 - Chaque objet informatique correspond à une entité réelle ou conceptuelle (ex. : voiture, compte bancaire, étudiant, produit...)
 - Chaque objet a :
 - ❖ Des caractéristiques → appelées attributs
 - ❖ Des comportements → appelés méthodes

UML – Programmation Orientée objet

❑ Code Programmation Orientée Objet :

```
class Voiture:
    def __init__(self, couleur, vitesse):
        self.couleur = couleur
        self.vitesse = vitesse

    def afficher(self):
        print("Voiture", self.couleur, "roule à", self.vitesse, "km/h")

ma_voiture = Voiture("rouge", 120)
ma_voiture.afficher()
```

❑ Code Programmation Procédural :

```
def afficher_voiture(couleur, vitesse):
    print("Voiture", couleur, "roule à", vitesse, "km/h")

couleur = "rouge"
vitesse = 120
afficher_voiture(couleur, vitesse)
```

UML – Programmation Orientée objet

❑ Programmation Orientée Objet VS Programmation Procédurale

Critère	Procédurale	Orientée Objet
Structure	Fonctions et variables séparées	Classes et objets
Organisation du code	Logique séquentielle	Modélisation du monde réel
Réutilisabilité	Faible (copie de code)	Forte (héritage, polymorphisme)
Maintenance	Plus difficile (dépendances globales)	Plus facile (modularité, encapsulation)
Adapté à	Petits scripts, calculs simples	Grands projets, systèmes complexes
Exemples de langages	C, Pascal	Java, C++, Python, C#, Ruby

UML – Programmation Orientée objet

- ❑ **Exemple :**
- ❑ marque et vitesse = attributs
- ❑ demarrer() et accélérer() = méthodes
- ❑ Voiture = classe (modèle)
- ❑ Une instance de Voiture = objet réel (par ex. maVoiture)

```
class Voiture {  
    String marque;  
    int vitesse;  
  
    void demarrer() {  
        System.out.println("La voiture démarre.");  
    }  
  
    void accélérer(int v) {  
        vitesse += v;  
    }  
}
```

UML – Programmation Orientée objet

❑ Les éléments constitutifs de la POO :

- ❑ Classe : est le plan de fabrication d'un objet. Elle décrit ce qu'un objet sait (attributs) et ce qu'il sait faire (méthodes).
- ❑ Attributs : Variables internes à la classe. Elles stockent l'état de l'objet.
- ❑ Méthodes : Fonctions internes à la classe. Elles définissent le comportement de l'objet.
- ❑ Objet : est une instance concrète d'une classe.
- ❑ Constructeur : méthode spéciale exécutée à la création de l'objet.

```
class Voiture {  
    String marque;  
    int vitesse;  
  
    void demarrer() {  
        System.out.println("La voiture démarre.");  
    }  
  
    void accélérer(int v) {  
        vitesse += v;  
    }  
}
```

```
Voiture v1 = new Voiture();  
v1.marque = "Renault";  
v1.vitesse = 100;  
v1.demarrer();
```

```
class Voiture {  
    String marque;  
    int vitesse;  
  
    Voiture(String m) {  
        marque = m;  
        vitesse = 0;  
    }  
}
```


UML – Programmation Orientée objet

❑ Valeurs de la Programmation Orientée Objet :

Concept	Définition	Exemple
Classe	Modèle ou plan de création d'objets	<code>class Voiture { ... }</code>
Objet	Instance concrète d'une classe	<code>maVoiture = new Voiture()</code>
Attributs	Variables internes à l'objet	<code>couleur, vitesse</code>
Méthodes	Fonctions internes à l'objet	<code>démarrer(), freiner()</code>
Encapsulation	Masquer les détails internes et protéger les données	<code>private vitesse; avec getVitesse()</code>
Héritage	Réutiliser le code d'une classe par une autre	<code>class Camion extends Voiture</code>
Polymorphisme	Une même méthode peut agir différemment selon l'objet	<code>afficher()</code> agit différemment pour Voiture et Camion
Abstraction	Simplifier la complexité en ne montrant que l'essentiel	Classe abstraite <code>Forme</code> avec <code>dessiner()</code>

UML – Programmation Orientée objet

- ❑ **Les piliers fondamentaux de la POO :**
- ❑ Encapsulation : protéger les données internes d'un objet pour éviter qu'elles soient modifiées n'importe comment. On rend les attributs privés (private) et on fournit des méthodes d'accès contrôlées (get et set).
- ❑ Héritage : permettre à une classe d'hériter des attributs et méthodes d'une autre. Cela évite la duplication de code.

```
class CompteBancaire {  
    private double solde;  
  
    public void depoter(double montant) {  
        solde += montant;  
    }  
  
    public double getSolde() {  
        return solde;  
    }  
}
```

```
class Personne {  
    String nom;  
    void parler() { System.out.println("Bonjour, je m'  
}  
  
class Etudiant extends Personne {  
    String filiere;  
}
```

UML – Programmation Orientée objet

❑ Les piliers fondamentaux de la POO :

- ❑ Polymorphisme : la même méthode peut avoir des comportements différents selon l'objet qui l'utilise. Ainsi, Le code devient plus flexible, on peut appeler parler() sans savoir le type exact d'animal.

```
Animal a1 = new Chien();
Animal a2 = new Chat();
a1.parler(); // "Le chien aboie"
a2.parler(); // "Le chat miaule"
```

- ❑ Abstraction : ne montrer que l'essentiel, cacher les détails techniques. Une classe abstraite définit un comportement commun, mais laisse les sous-classes décider de l'implémentation.

```
class Animal {
    void parler() { System.out.println("L'animal fait")
}

class Chien extends Animal {
    void parler() { System.out.println("Le chien aboie")
}

class Chat extends Animal {
    void parler() { System.out.println("Le chat miaule")
}
```

```
abstract class Forme {
    abstract void dessiner();
}

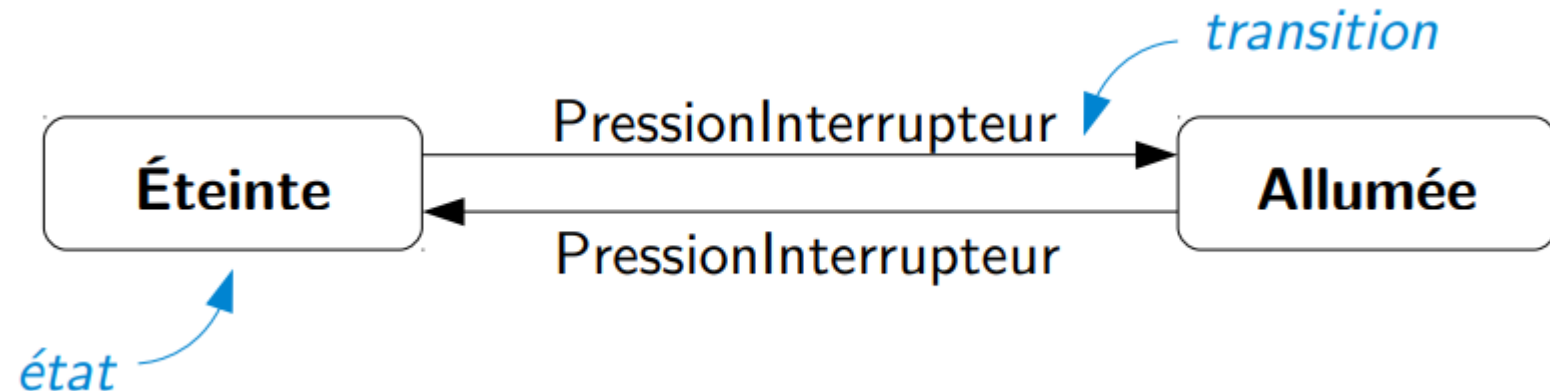
class Cercle extends Forme {
    void dessiner() { System.out.println("Je dessine u")
}
```

UML – Diagramme d'états-transition

- ❑ C'est le diagramme UML qui décrit le cycle de vie d'un **objet**/système réactif : dans quel état il se trouve, quels événements le font changer d'état, et quelles actions sont exécutées en cours de route.
- ❑ Modéliser des comportements réactifs : interfaces, workflows, IoT, protocoles, automates, règles métier séquentielles.
- ❑ Clarifier les règles de passage d'un état à un autre (qui a le droit, quand, sous quelles conditions).
- ❑ Détecter les trous de logique : transitions manquantes, boucles impossibles, conflits de règles.
- ❑ Comportement décrit par états + transitions entre les états
 - **État** : abstraction d'un moment de la vie d'une entité pendant lequel elle satisfait un ensemble de conditions
 - **Transition** : changement d'état

UML – Diagramme d'états-transition

- ❑ Comportement décrit par états + transitions entre les états
 - **État** : abstraction d'un moment de la vie d'une entité pendant lequel elle satisfait un ensemble de conditions
 - **Transition** : changement d'état



- ❑ Intérêt :
 - Vue synthétique de la dynamique de l'entité
 - Regroupe un ensemble de scénarios

UML – Diagramme d'états-transition

❑ Types d'états :

- **État initial** : Initialisation du système, exécution du constructeur de l'objet.
- **État final** : Fin de vie du système, destruction de l'objet
- **États intermédiaires** : étapes de la vie du système, de l'objet



ÉtatSimple

ÉtatAvecEvt

event1 [cond1] / action1
event2 [cond2] / action2

- ❑ **Événement** : Fait instantané venant de l'extérieur du système et survenant à un instant donné. Un événement se produit à un instant précis et est dépourvu de durée. Quand un événement est reçu, une transition peut être déclenchée et faire basculer l'objet dans un nouvel état. On peut diviser les événements en plusieurs types explicites et implicites : signal, appel, changement et temporel.

UML – Diagramme d'états-transition

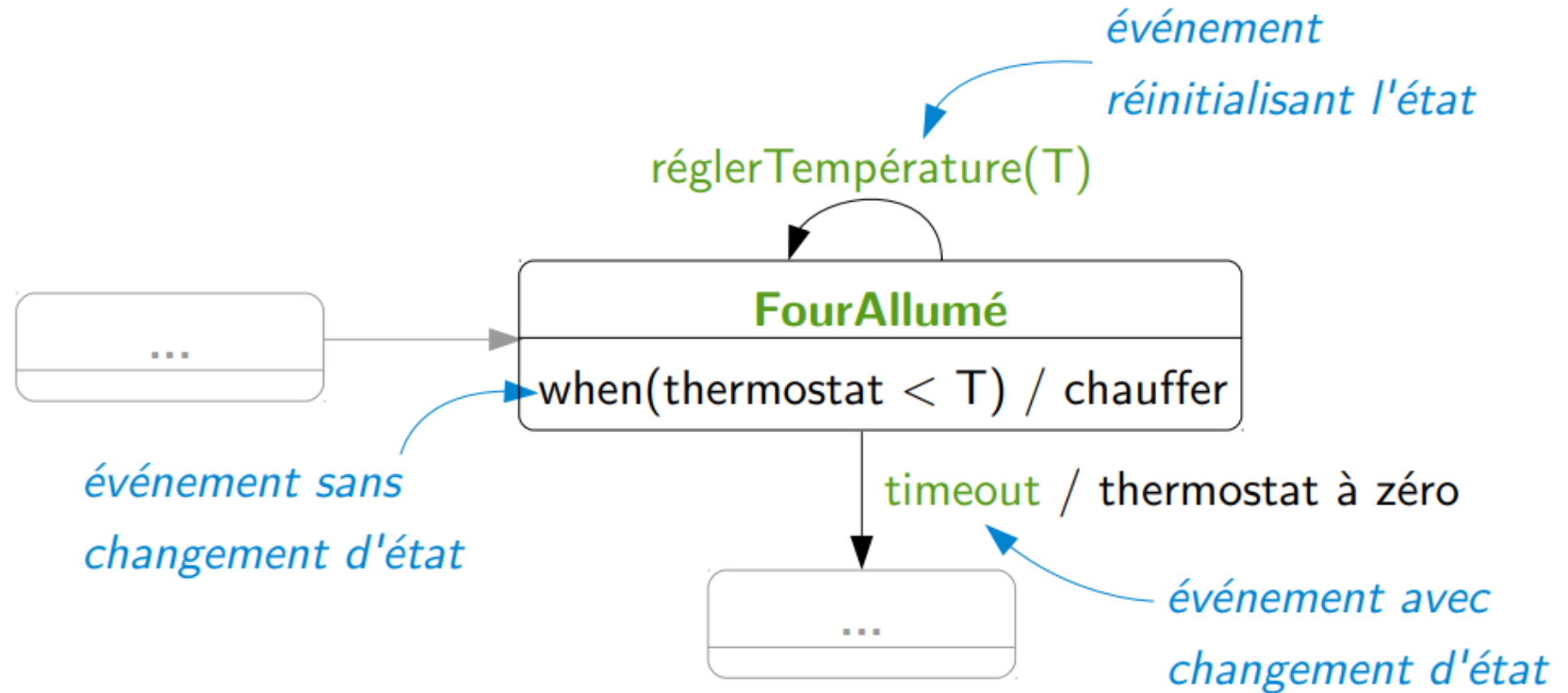
□ Types d'événements :

- **Signal** : réception d'un message asynchrone
- **Appel d'une opération (synchrone)** : liée aux cas d'utilisation...
- **Satisfaction d'une condition booléenne** : when(cond), évaluée continuellement jusqu'à ce qu'elle soit vraie
- **Temps** :
 - ❖ Date relative : when(date = date)
 - ❖ Date absolue : after(durée)

UML – Diagramme d'états-transition

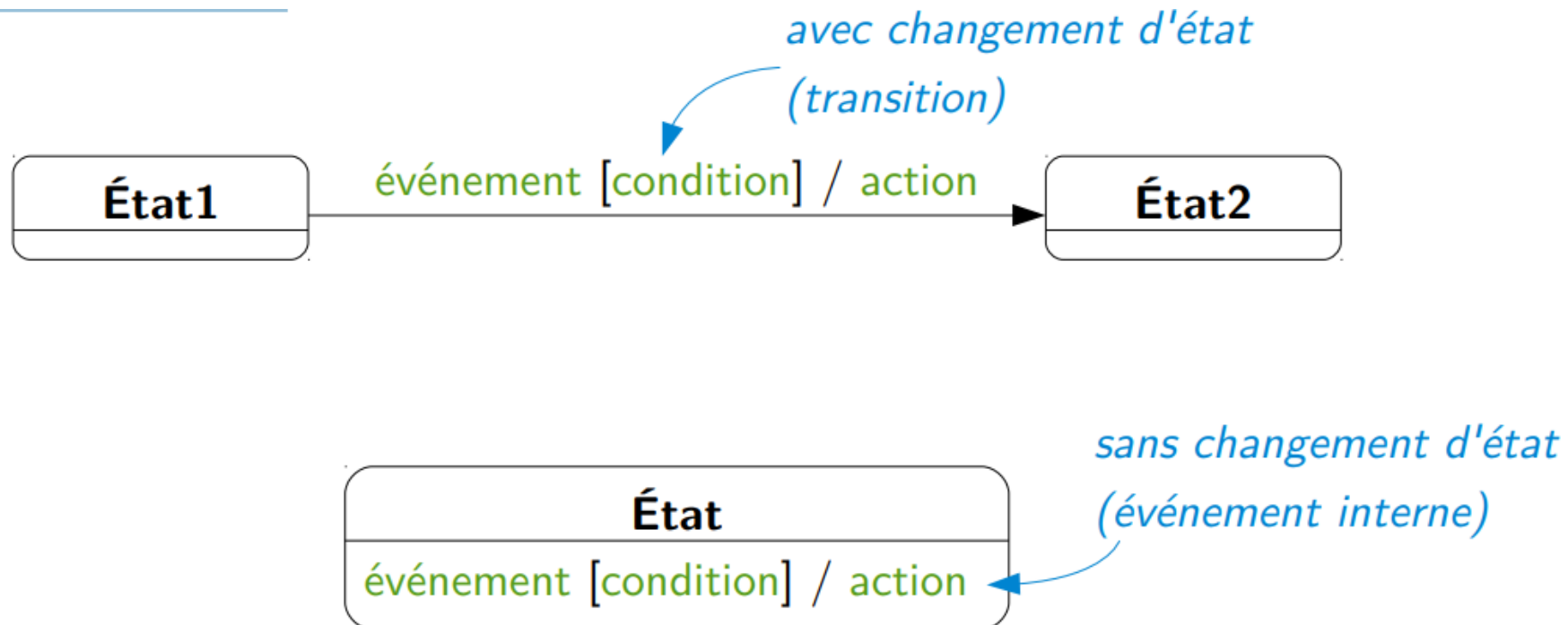
❑ Caractéristiques d'un état :

- Conditions vérifiées
- Événements attendus



UML – Diagramme d'états-transition

- ❑ **Action** : Réaction du système à un événement. Elle est atomique, instantanée, non interruptible.
- ❑ Exemples d'actions (syntaxe laissée libre) : affectation, envoi d'un signal, appel d'une opération, création ou destruction d'un objet.
- ❑ Action déclenchée par un événement **événement [condition] / action** Lorsque l'événement se produit, si la condition est vérifiée, alors l'action est effectuée

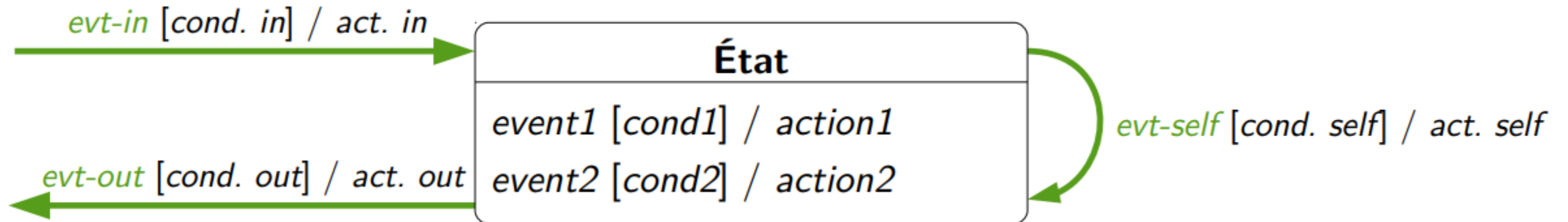


UML – Diagramme d'états-transition

❑ **Événements internes à l'état** : Événement sans changement d'état.

❑ **Événements externes à l'état** : transitions

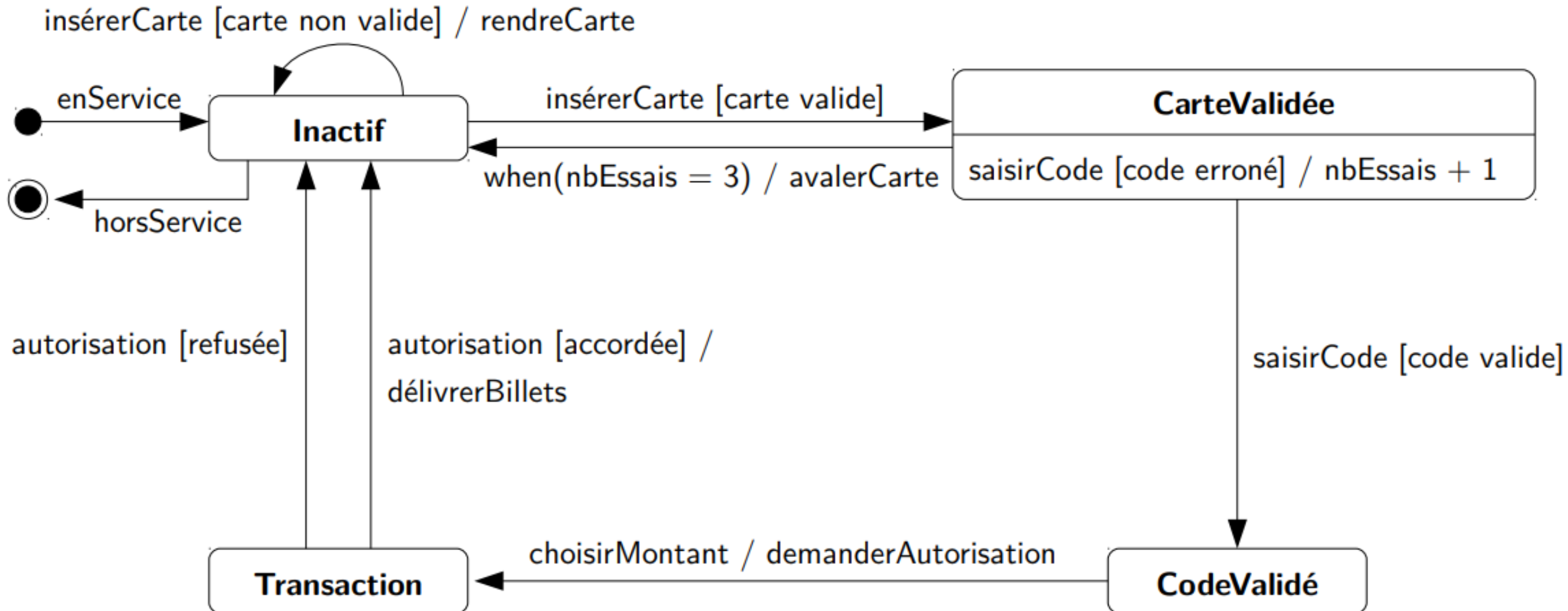
- Transition vers l'état : evt-in
- Transition depuis l'état : evt-out
- Transition depuis l'état vers lui-même : evt-self



État
<i>event1 [cond1] / action1</i>
<i>event2 [cond2] / action2</i>

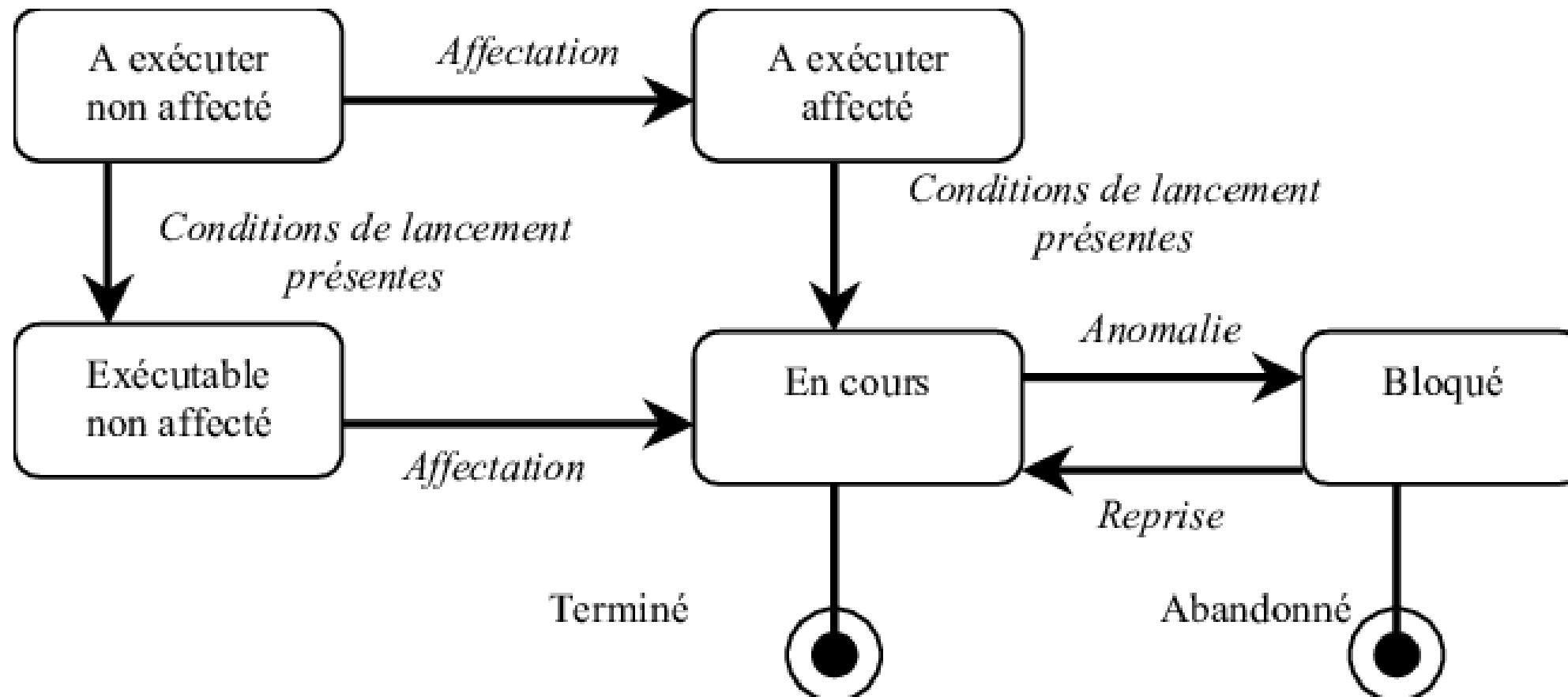
UML – Diagramme d'états-transition

Exemple diagramme état transition de l'objet **DBA** :



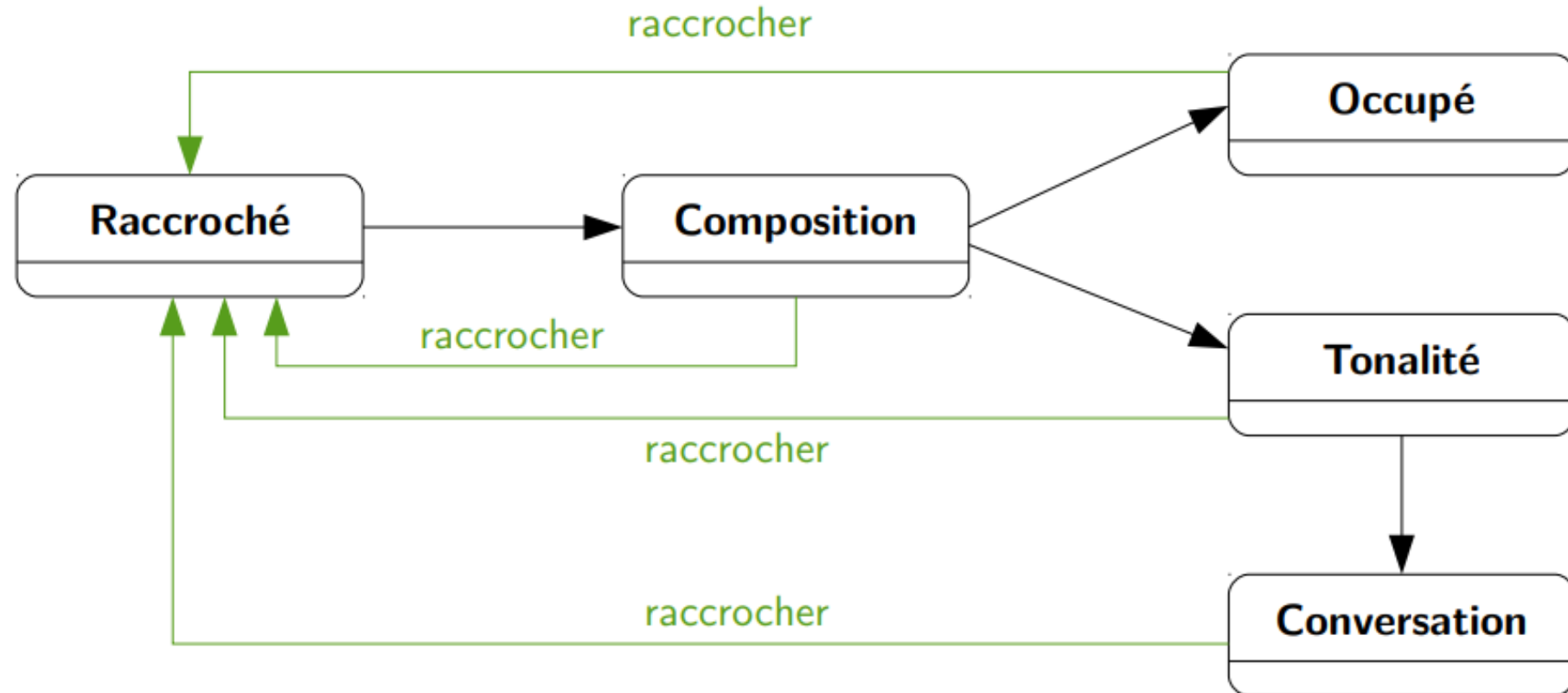
UML – Diagramme d'états-transition

❑ Exemple diagramme état transition de l'objet **Tâche** :



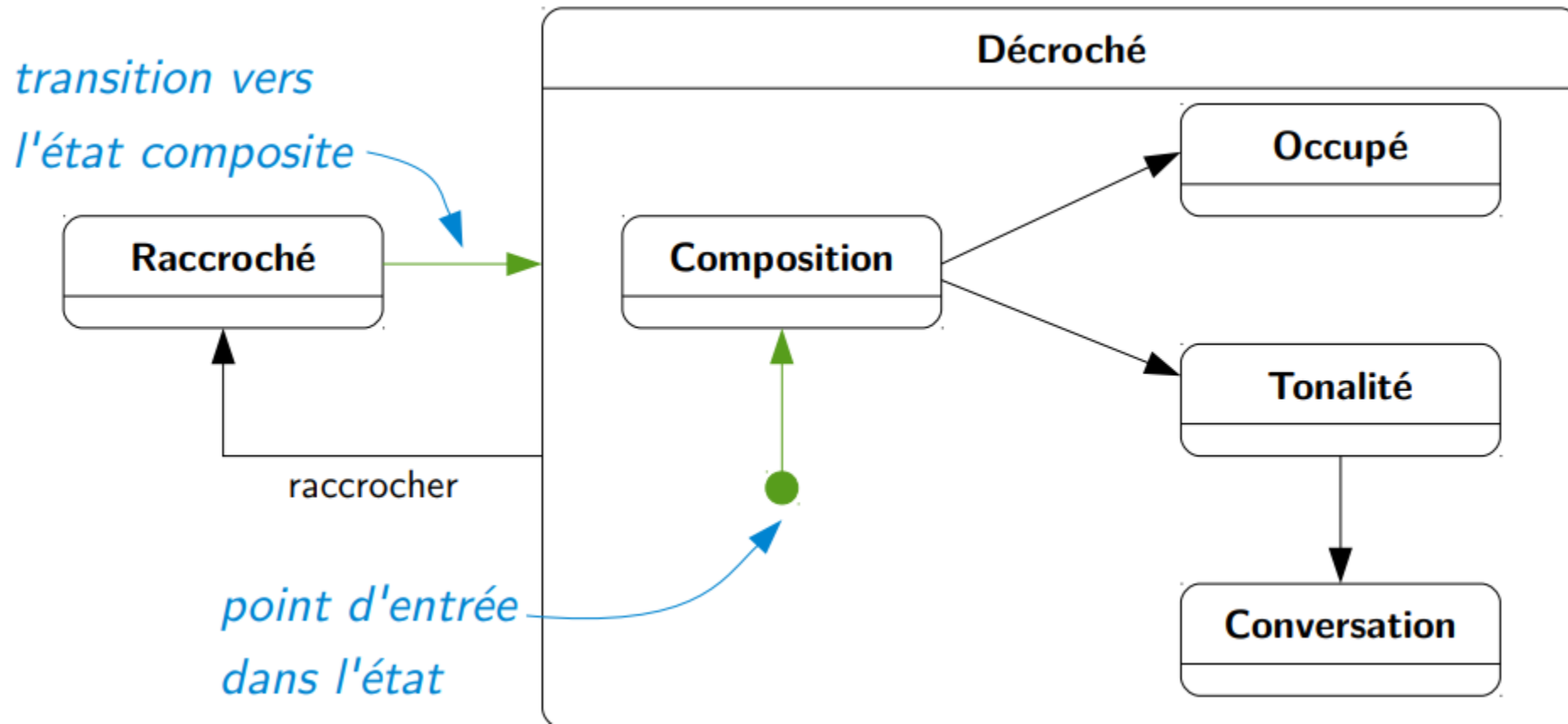
UML – Diagramme d'états-transition

- ❑ **Etat composite** : État regroupant un ensemble d'états dans l'objectif
 - Hiérarchiser les états
 - Structurer les comportements complexes
 - Factoriser les actions



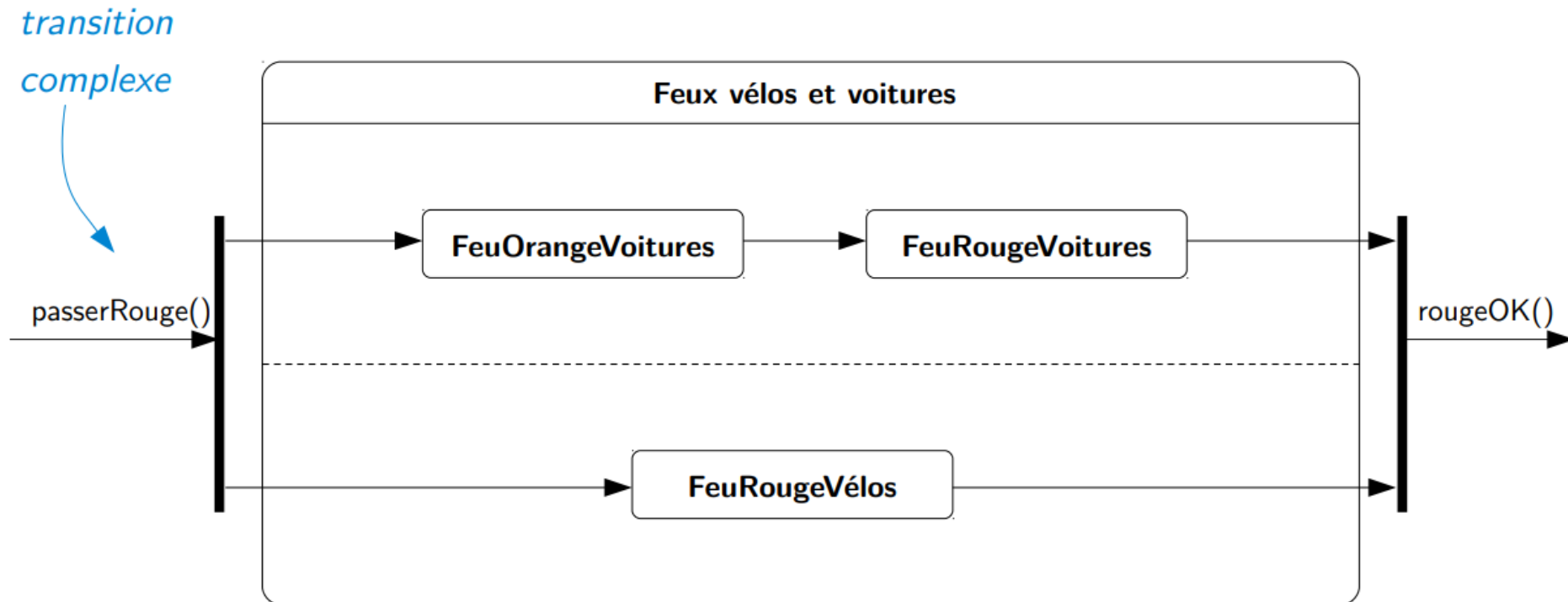
UML – Diagramme d'états-transition

- ❑ **Etat composite** : État regroupant un ensemble d'états dans l'objectif
 - Hiérarchiser les états
 - Structurer les comportements complexes
 - Factoriser les actions



UML – Diagramme d'états-transition

- ❑ **Etat orthogonal** : État composite dans lequel plusieurs états sont actifs simultanément (concurrency/parallélisme)
- Hiérarchiser les états
- Structurer les comportements complexes
- Factoriser les actions



UML – Diagramme d'états-transition

❑ Exercice d'application

Une bibliothèque municipale souhaite informatiser la gestion de l'emprunt de livres.

Un usager peut :

- Consulter le catalogue en ligne.
- S'inscrire comme membre.
- Emprunter un livre s'il est membre et que le livre est disponible.
- Retourner un livre.

Un bibliothécaire peut :

- Enregistrer l'inscription d'un nouvel usager.
 - Enregistrer les emprunts et les retours de livres.
 - Mettre à jour le catalogue (ajouter/supprimer des livres).
-
- Définir au moins un objet qui change d'états dans le système.
 - Réaliser le diagramme d'états-transitions relatif à cet objet.